Universite Cadi Ayyad

Ecole Nationale des Sciences Appliquees de Marrakech

Genie Cyber Defense et Systemes de Telecommunications Embarques

Rapport de Projet de Fin d'Annee

Plateforme de Gouvernance Dynamique du Risque Vulnerabilite

Modelisation, ingestion et gouvernance dynamique du risque cyber a partir des donnees Qualys et CMDB

Realise par :
Mohamed Amine Faik

Encadre par :
[Nom de l'encadrant academique] (academique)
[Nom de l'encadrant professionnel] (professionnel)

Organisme d'accueil :
DXC Technology

Soutenu le :
[date de soutenance]

Annee universitaire : 2025 – 2026

A mes parents, pour leur soutien constant,
a mes enseignants, pour la rigueur qu'ils m'ont transmise,
et a toute personne ayant contribue, de pres ou de loin,
a la reussite de ce projet.

Remerciements

Je tiens tout d'abord a exprimer ma sincere gratitude a mon encadrant professionnel, [Nom de l'encadrant professionnel], pour sa disponibilite, ses conseils techniques avises et la confiance qu'il m'a accordee tout au long de ce stage au sein de DXC Technology.

Mes remerciements s'adressent egalement a mon encadrant academique, [Nom de l'encadrant academique], pour son suivi regulier, ses recommandations methodologiques et la pertinence de ses retours qui ont grandement contribue a la qualite de ce travail.

Je remercie aussi l'ensemble des equipes de DXC Technology, dont l'accueil et le partage d'expertise en cybersecurite et en ingenierie logicielle m'ont permis de mener ce projet dans les meilleures conditions.

Enfin, je remercie les membres du jury pour l'interet qu'ils portent a ce travail en acceptant de l'evaluer, ainsi que toute personne ayant, de pres ou de loin, participe a l'aboutissement de ce projet de fin d'annee.

Resume

Les organisations exposees a Internet accumulent des milliers de vulnerabilites detectees par des scanners tels que Qualys, sans toujours disposer d'un moyen fiable de les hierarchiser selon le risque reel qu'elles font courir au systeme d'information. Une severite technique brute (CVSS ou QDS) ne suffit pas : un meme defaut n'a pas le meme impact selon qu'il touche un serveur de test isole ou une base de donnees de production exposee sur Internet. Ce projet, realise dans le cadre d'un stage de fin d'annee au sein de DXC Technology, propose une plateforme qui croise automatiquement les exports Qualys (vulnerabilites) et CMDB (criticite metier des actifs) pour calculer un score de risque contextualise par actif, l'*Asset Risk Score* (ARS), et pour en deriver une gouvernance operationnelle (classes de risque, delais de remediation SLA, mesure de la dette operationnelle via le MTTR pondere).

Le travail effectue couvre l'ensemble de la chaine applicative : l'ingestion asynchrone et par lots (Spring Batch) des fichiers XML Qualys et CMDB avec dedoublonnage par empreinte cryptographique, la modelisation mathematique et l'implementation du moteur de calcul de risque, la construction d'un tableau de bord temps reel (Server-Sent Events, Chart.js) restituant la posture de securite du parc sous forme de graphiques (repartition par classe, diagramme de Sankey, historique, conformite SLA), ainsi qu'un assistant conversationnel fonde sur une architecture RAG (Retrieval-Augmented Generation) permettant d'interroger ces donnees en langage naturel – ce dernier volet restant, au moment de la redaction, au stade du developpement et de la conception (prototype fonctionnel non encore valide pour un environnement de recette ou de production). Ce rapport detaille le contexte, la conception, les fondements mathematiques – demontres formellement – du modele de risque, la realisation technique et les perspectives d'evolution de la plateforme.

Mots-cles : gestion des vulnerabilites, risque cyber, CMDB, Qualys, Asset Risk Score, MTTR, Spring Batch, tableau de bord temps reel, RAG, Spring Boot.

Abstract

Organisations exposed to the Internet accumulate thousands of vulnerabilities detected by scanners such as Qualys, without always having a reliable way to prioritise them according to the actual risk they pose to the information system. A raw technical severity score (CVSS or QDS) is not sufficient: the same flaw does not carry the same impact depending on whether it affects an isolated test server or an Internet-facing production database. This project, carried out as part of a final year internship at DXC Technology, introduces a platform that automatically cross-references Qualys exports (vulnerabilities) with CMDB exports (business criticality of assets) to compute a contextualised risk score per asset, the *Asset Risk Score* (ARS), and to derive an operational governance layer from it (risk classes, SLA remediation deadlines, operational debt measurement through a weighted MTTR).

The work covers the whole application chain: asynchronous, batch-oriented ingestion (Spring Batch) of Qualys and CMDB XML files with deduplication by cryptographic hash, the mathematical modelling and implementation of the risk-scoring engine, a real-time dashboard (Server-Sent Events, Chart.js) rendering the fleet's security posture through charts (risk-class distribution, Sankey diagram, historical trend, SLA compliance), and a conversational assistant built on a Retrieval-Augmented Generation (RAG) architecture allowing this data to be queried in natural language – this last component still being, at the time of writing, at the development and design stage (a functional prototype not yet validated for a staging or production environment). This report details the context, the design, the mathematically proven foundations of the risk model, the technical implementation and the future perspectives of the platform.

Keywords: vulnerability management, cyber risk, CMDB, Qualys, Asset Risk Score, MTTR, Spring Batch, real-time dashboard, RAG, Spring Boot.

Table des matières

Remerciements

Resume

Abstract

Liste des abreviations **vi**

Introduction generale **1**

1 Contexte general du projet **3**
1.1 Organisme d'accueil . . . 3
1.2 Problematique . . . 3
1.3 Objectifs du projet . . . 4
1.4 Perimetre du projet . . . 4
1.4.1 Ce qui est couvert . . . 4
1.4.2 Ce qui est explicitement hors perimetre . . . 5
1.5 Methodologie de travail . . . 5
1.6 Organisation du reste du rapport . . . 6

2 Analyse et specification des besoins **7**
2.1 Identification des acteurs . . . 7
2.2 Besoins fonctionnels . . . 7
2.3 Besoins non fonctionnels . . . 8
2.4 Diagramme de cas d'utilisation . . . 9
2.5 Description textuelle des cas d'utilisation principaux . . . 10
2.5.1 UC – Importer un rapport Qualys / un export CMDB . . . 10
2.5.2 UC – Consulter le tableau de bord temps reel . . . 11
2.5.3 UC – Interroger l'assistant conversationnel (RAG) . . . 11

3 Conception et architecture du systeme **13**
3.1 Architecture globale . . . 13
3.2 Modele de donnees – diagramme de classes . . . 14
3.2.1 Le choix de la reconciliation par cle applicative . . . 15

3.3 Diagrammes de sequence 16
3.3.1 Ingestion d'un export et recalcul du risque 16
3.3.2 Consultation de l'assistant conversationnel 17
3.4 Choix technologiques 18

4 Modelisation mathematique du risque 20
4.1 Notations 20
4.2 Le score de criticite d'actif (ACS) 21
4.2.1 Definition 21
4.3 Le score de vulnerabilite pondere par severite 21
4.3.1 Formule de reference 21
4.3.2 Raffinement implemente 22
4.4 L'Asset Risk Score (ARS) 23
4.4.1 Equation fondamentale 23
4.4.2 Positivite et monotonie 24
4.4.3 Condition de saturation 24
4.5 Analyse de sensibilite : la hierarchie des severites 25
4.6 Classification en classes de risque et gouvernance SLA 26
4.6.1 Classification 26
4.6.2 Gestion des exceptions (Risk Acceptance) 27
4.7 Mesure de la performance : le MTTR pondere 27
4.7.1 Definition 27
4.8 Agregation globale : l'Enterprise Risk Score (ERS) 28
4.8.1 Definition 28
4.8.2 Ecart entre specification et implementation actuelle 29
4.9 Synthese 30

5 Realisation et implementation 31
5.1 Environnement technique 31
5.2 Pipeline d'ingestion des donnees 31
5.2.1 Reception et securisation du fichier 31
5.2.2 Traitement par lots (Spring Batch) 32
5.2.3 Recalcul du risque : parallelisme et coherence transactionnelle . 33
5.3 Tableau de bord temps reel 33
5.4 Assistant conversationnel (RAG) 34
5.5 Conteneurisation et deploiement 34

6 Visualisation des donnees : lecture et interpretation des graphiques 35
6.1 Repartition par classe de risque 35
6.2 Repartition par equipe proprietaire 36

6.3 Flux Environnement → Role → Classe de risque 36
6.4 Historique de la repartition du parc 37
6.5 Conformite SLA (age vs. score de risque) 38

7 Tests, validation et resultats 40
7.1 Strategie de test 40
7.2 Validation numerique du modele ARS 40
7.3 Validation du MTTR pondere 42
7.4 Bilan des resultats obtenus 42

Conclusion generale et perspectives 43
Bilan 43
Limites du travail realise 43
Perspectives 44

A Extraits des formats de donnees en entree 46
A.1 Extrait d'un export CMDB (XML) 46
A.2 Extrait d'un rapport Qualys (XML) 46

B Parametres par defaut du modele de risque 48

C Glossaire 49

Bibliographie 50

Table des figures

2.1 Diagramme de cas d'utilisation de la plateforme 10

3.1 Diagramme de deploiement (conteneurs Docker Compose) 13
3.2 Architecture logicielle en couches 14
3.3 Diagramme de classes simplifie du domaine 15
3.4 Diagramme de sequence – ingestion d'un export et recalcul du risque . 17
3.5 Diagramme de sequence – interrogation de l'assistant RAG 18

6.1 Repartition des actifs par classe de risque (diagramme en anneau, donnees illustratives) 35
6.2 Repartition des actifs par equipe proprietaire (barres horizontales empilees, donnees illustratives) 36
6.3 Diagramme de flux (Sankey) Environnement → Role → Classe de risque (donnees illustratives) 37
6.4 Historique empile de la repartition par classe (donnees illustratives, une serie par instantane `RiskSnapshot`) 38
6.5 Conformite SLA : anciennete vs. ARS, avec zones de classe en fond (donnees illustratives) 39

Liste des tableaux

4.2 Matrice des multiplicateurs de risque residuel 24
4.3 Matrice dynamique des SLA de remediation (valeurs de reference) . . . 26
4.4 Recapitulatif des formules du modele de risque 30

B.1 Valeurs par defaut de la configuration du risque 48

Liste des abreviations

Sigle	Signification
ACS	Asset Criticality Score (score de criticite metier de l'actif)
ARS	Asset Risk Score (score de risque de l'actif)
ERS	Enterprise Risk Score (score de risque agrege de l'entreprise)
QDS	Qualys Detection Score (score de detection Qualys, 0–100)
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CMDB	Configuration Management Database
CI	Configuration Item (element de configuration dans la CMDB)
SLA	Service Level Agreement (delai de remediation exige)
MTTR	Mean Time To Remediate / Repair (temps moyen de remediation)
MTTR_w	MTTR pondere par classe de risque
SOC	Security Operations Center
EDR	Endpoint Detection and Response
SIEM	Security Information and Event Management
RAG	Retrieval-Augmented Generation
LLM	Large Language Model (grand modele de langage)
SSE	Server-Sent Events
DTO	Data Transfer Object
JPA	Java Persistence API
UML	Unified Modeling Language
XML	eXtensible Markup Language
STAX	Streaming API for XML

Sigle	**Signification**
HTMX	bibliotheque JavaScript de rendu HTML dynamique cote serveur

Introduction generale

Dans un systeme d'information moderne, le nombre de vulnerabilites remontees chaque semaine par un scanner comme Qualys se compte en milliers. Chaque vulnerabilite est associee a un score technique – le QDS (Qualys Detection Score) – qui mesure la gravite intrinseque de la faille, independamment de l'actif sur lequel elle a ete detectee. Or, un serveur de developpement isole du reseau et une base de donnees de production accessible depuis Internet ne presentent pas le meme risque pour l'entreprise, meme s'ils partagent exactement les memes vulnerabilites. Traiter toutes les alertes avec la meme priorite revient soit a noyer les equipes de remediation sous un volume ingerable, soit, a l'inverse, a laisser filer des risques critiques faute de hierarchisation.

C'est ce probleme – passer d'un inventaire brut de vulnerabilites a une hierarchisation actionnable du risque – que ce projet de fin d'annee cherche a resoudre. Realise au sein de DXC Technology, il a donne naissance a une plateforme qui croise deux sources de donnees historiquement traitees separement : les exports de scan Qualys (la probabilite d'exploitation technique) et la CMDB (la criticite metier et les mesures de protection deja en place sur chaque actif). De ce croisement est derive un score unique et explicable par actif, l'*Asset Risk Score*, qui pilote ensuite des delais de remediation differencies (SLA) et permet de mesurer objectivement la performance des equipes charge es de corriger les failles (MTTR pondere).

Ce rapport documente l'ensemble de la demarche suivie. Le premier chapitre situe le projet dans son contexte organisationnel et technique, formule la problematique et delimite precisement son perimetre. Le deuxieme chapitre specifie les besoins fonctionnels et non fonctionnels de la plateforme a travers une analyse UML des cas d'utilisation. Le troisieme chapitre presente la conception logicielle retenue : architecture globale, modele de donnees et diagrammes de sequence des principaux flux. Le quatrieme chapitre, coeur scientifique du projet, formalise et demontre mathematiquement le modele de risque – ARS, classification SLA, MTTR pondere et agregation en un Enterprise

Risk Score. Le cinquieme chapitre decrit la realisation technique de la plateforme, de l'ingestion des donnees a l'assistant conversationnel. Le sixieme chapitre explique, graphique par graphique, comment lire le tableau de bord produit par l'application. Le septieme chapitre presente la strategie de test et valide numeriquement le modele sur un exemple complet. Le rapport se conclut par un bilan critique du travail realise et par les perspectives d'evolution envisagees pour la plateforme.

1 Contexte general du projet

1.1 Organisme d'accueil

Ce projet a ete realise dans le cadre d'un stage de fin d'annee au sein de DXC Technology [1]. DXC Technology est une entreprise de services numeriques qui accompagne ses clients dans la modernisation, la securisation et l'exploitation de leurs systemes d'information. Le stage s'est deroule au sein d'une equipe technique en charge de la securite operationnelle (*DevSecOps*), dont l'une des missions est de fournir aux equipes clientes une visibilite fiable et exploitable sur l'exposition aux vulnerabilites de leur parc informatique.

1.2 Problematique

Le point de depart du projet est un constat operationnel, formalise dans les notes de cadrage initiales (voir *app_notes_dxc.txt*) : les donnees necessaires a l'evaluation du risque existent, mais elles sont fragmentees et de natures tres differentes.

- Les exports **Qualys** decrivent, pour chaque hote scanne, la liste des vulnerabilites detectees (identifiant QID, severite, score QDS, dates de premiere et derniere detection, statut de correction). Ces donnees sont *volumineuses* et *mises a jour frequemment* (un nouveau scan peut redefinir l'etat complet du parc).
- La **CMDB** (Configuration Management Database) decrit, pour chaque element de configuration, son role technique, son environnement (production, test, developpement...), son proprietaire metier et les mesures de protection deployees (antivirus/EDR, supervision SOC, cycle de patch standard). Ces donnees sont *plus stables* mais *lues frequemment* par tous les traitements de scoring.

1. **[A completer : presentation de l'agence/departement d'accueil, activites, effectif, positionnement sur le marche de la cybersecurite et des services IT.]**

Avant ce projet, le rapprochement entre ces deux sources – indispensable pour distinguer une vulnerabilite critique sur un serveur de production expose d'une vulnerabilite identique sur un poste de test isole – se faisait manuellement (feuille de calcul), ce qui est lent, source d'erreurs, non tracable dans le temps et ne permet aucune restitution visuelle consolidee pour le pilotage. Cette absence de score de risque contextualise et de gouvernance SLA associee constitue la problematique centrale adressee par ce projet.

1.3 Objectifs du projet

Les objectifs assignes au stage, tels qu'ils decoulent du cadrage initial, sont les suivants :

1. Concevoir un modele mathematique de risque qui combine la criticite metier d'un actif (issue de la CMDB), la gravite intrinseque de ses vulnerabilites (issue de Qualys), son exposition reseau et ses controles compensatoires, en un score unique, explicable et reproductible : l'*Asset Risk Score* (ARS).

2. Automatiser l'ingestion des exports Qualys et CMDB (fichiers XML), avec detection des doublons et traitement asynchrone, pour supporter des mises a jour frequentes sans intervention manuelle.

3. Deriver de l'ARS une gouvernance operationnelle : classification en 5 classes de risque (*Green* a *Black*), delais de remediation SLA associes, et gestion des exceptions de risque accepte.

4. Mesurer objectivement la performance des equipes de remediation via un indicateur de temps moyen de correction pondere par criticite (MTTR pondere).

5. Restituer l'ensemble de ces informations dans un tableau de bord temps reel, exploitable par un RSSI ou un analyste securite, combinant plusieurs types de graphiques (repartition, flux, historique, conformite SLA).

6. Offrir un canal d'interrogation en langage naturel de ces donnees via un assistant conversationnel fonde sur une architecture RAG.

1.4 Perimetre du projet

1.4.1 Ce qui est couvert

- L'ingestion, le parsing et la persistance des exports Qualys (rapport de vulnerabilites par hote) et CMDB (inventaire des elements de configuration), au format XML.

- Le calcul, pour chaque actif reconcilie (present a la fois dans Qualys et dans la CMDB), de l'ACS, du score QDS pondere par severite, de l'ARS, de la classe de risque, du statut SLA et de l'anciennete de la vulnerabilite ouverte la plus ancienne.
- La configuration, par l'utilisateur, des poids du modele : poids par severite, poids par role technique et par environnement (pour l'ACS), facteurs de modulation ($I_{External}$, M_{SOC}, M_{AV}, M_{Patch}), seuils et delais SLA par classe, poids de ponderation du MTTR par classe.
- Un tableau de bord de pilotage alimente en temps reel (Server-Sent Events) presentant la repartition du parc par classe de risque, par equipe proprietaire, les flux Environnement/Role/Classe (diagramme de Sankey), l'historique de l'ARS moyen et la conformite SLA (age des vulnerabilites ouvertes vs. ARS).
- Un assistant conversationnel RAG capable de repondre, en langage naturel, a des questions portant sur les donnees ingerees et calculees – livre sous forme de prototype fonctionnel, ce module restant au stade du developpement et de la conception (cf. limites, section 7.4).

1.4.2 Ce qui est explicitement hors perimetre

- L'agregation multi-scan dans le temps au niveau de la CMDB : chaque nouvel import remplace integralement l'etat precedent (strategie *wipe-and-reingest*, cf. section 5.2) ; il n'y a pas, dans la version realisee, de gestion de versions historiques des CI.
- L'authentification et le controle d'acces par role (RBAC) : la version realisee est un prototype fonctionnel dont la chaine de securite HTTP est volontairement ouverte (voir limites, section 7.4).
- La connexion a d'autres sources de vulnerabilites que Qualys (Nessus, Tenable, OpenVAS...) ou a d'autres CMDB que le format d'export etudie.
- Le calcul effectif du facteur de penalite operationnelle Global_I_{Ops} dans l'agregation *Enterprise Risk Score* : le modele mathematique est entierement specifie et demontre (chapitre 4) mais son integration dans le service applicatif reste une perspective d'evolution.

1.5 Methodologie de travail

Le developpement a suivi une demarche iterative et incrementale, guidee par l'historique des livraisons du projet : une premiere version couvrant l'ingestion et le modele

de donnees de base, suivie d'une phase de fiabilisation de l'ingestion CMDB, puis d'une extension fonctionnelle majeure (moteur de risque configurable, tableau de bord temps reel, assistant RAG), et enfin d'une phase de conteneurisation en vue du deploiement (Docker Compose). Chaque etape a fait l'objet de commits Git dedies, permettant une tracabilite complete des choix d'implementation. Cette approche a permis de valider tot le pipeline de bout en bout (upload → ingestion → calcul → restitution) avant d'en enrichir progressivement la richesse fonctionnelle (configuration dynamique des poids, gestion des exceptions, historisation des instantanes de risque).

1.6 Organisation du reste du rapport

La suite du rapport suit la logique amont-aval d'un projet logiciel : specification des besoins (chapitre 2), conception (chapitre 3), fondements mathematiques du coeur metier (chapitre 4), realisation (chapitre 5), restitution visuelle (chapitre 6) et validation (chapitre 7), avant de conclure sur le bilan et les perspectives.

2 Analyse et specification des besoins

2.1 Identification des acteurs

L'analyse du perimetre fonctionnel (section 1.4) fait apparaitre trois acteurs interagissant avec la plateforme :

- **Analyste securite / RSSI** : consulte le tableau de bord, explore le detail du score d'un actif, interroge l'assistant conversationnel. C'est l'utilisateur principal du systeme au quotidien.
- **Administrateur fonctionnel** : en plus des actions de l'analyste, importe les nouveaux exports Qualys/CMDB et configure les parametres du modele de risque (poids, facteurs, seuils SLA). Dans le modele UML, cet acteur *generalise* l'analyste : tout ce que l'analyste peut faire, l'administrateur le peut aussi.
- **Moteur LLM local (Ollama)** : acteur secondaire, externe au systeme, sollicite par le cas d'utilisation « Interroger l'assistant RAG » pour generer une reponse en langage naturel a partir des donnees recuperees par les outils exposes a l'agent.

2.2 Besoins fonctionnels

Code	Besoin fonctionnel
BF-01	Importer un fichier d'export Qualys (XML) et en extraire la liste des hotes et de leurs vulnerabilites.
BF-02	Importer un fichier d'export CMDB (XML) et en extraire la liste des elements de configuration (CI).

Code	Besoin fonctionnel
BF-03	Rejeter tout fichier deja traite (detection par empreinte cryptographique) et tout fichier dont le type MIME reel ne correspond pas au type declare.
BF-04	Recalculer automatiquement, des qu'un import aboutit ou qu'une configuration change, l'ACS, le score QDS pondere, l'ARS, la classe de risque et le statut SLA de chaque actif reconcilie.
BF-05	Permettre a l'administrateur de configurer les poids de severite (w_c, w_h, w_m, w_l), les poids par role technique et par environnement, les facteurs de modulation ($I_{External}$, M_{SOC}, M_{AV}, M_{Patch}) et les seuils/delais SLA par classe de risque.
BF-06	Marquer un actif en exception (« risque accepte ») et refleter cet etat dans le statut SLA affiche, sans alterer l'ARS calcule.
BF-07	Afficher un tableau de bord synthetisant la repartition du parc par classe de risque, par equipe/proprietaire, les flux Environnement/Role/Classe, l'historique de l'ARS moyen et la conformite SLA.
BF-08	Mettre a jour le tableau de bord en temps reel (sans rechargement de page) des qu'un recalcul de risque aboutit.
BF-09	Afficher, pour un actif donne, le detail explicatif du calcul de son ARS (contribution de chaque terme de la formule).
BF-10	Permettre a l'analyste d'interroger en langage naturel les donnees ingerees et calculees via un assistant conversationnel.

2.3 Besoins non fonctionnels

Categorie	Exigence
Performance	Le calcul de l'ARS pour l'ensemble du parc doit s'effectuer en parallele (un couple ACS/QDS pondere par actif) pour rester compatible avec des imports comportant plusieurs milliers d'hotes.
Reactivite	Le tableau de bord doit refleter un nouveau recalcul sans action de l'utilisateur (push serveur → client).

Categorie	Exigence
Coherence transactionnelle	Un evenement declenchant un recalcul ne doit etre traite qu'apres la validation (*commit*) de la transaction qui l'a emis, afin d'eviter de lire un etat de base de donnees partiel.
Idempotence des imports	Un meme fichier ne doit jamais etre traite deux fois ; un nouvel import d'un meme type de source remplace integralement l'etat precedent.
Robustesse des entrees	Le type reel d'un fichier uploade doit etre verifie independamment de son extension declaree (protection contre l'upload de fichiers non maitrise).
Configurabilite	Aucun parametre du modele de risque (poids, facteurs, seuils) ne doit etre code en dur : tous doivent rester modifiables sans redeploiement.
Portabilite	L'application doit pouvoir etre deployee de maniere reproductible via conteneurisation (Docker Compose), y compris le moteur d'inference du LLM local.

2.4 Diagramme de cas d'utilisation

Le figure 2.1 synthetise les besoins fonctionnels precedents sous forme de diagramme de cas d'utilisation UML. Les relations `«include»` traduisent le fait que trois evenements distincts (import Qualys, import CMDB, soumission d'une configuration) declenchent systematiquement le meme sous-processus : le recalcul du risque.

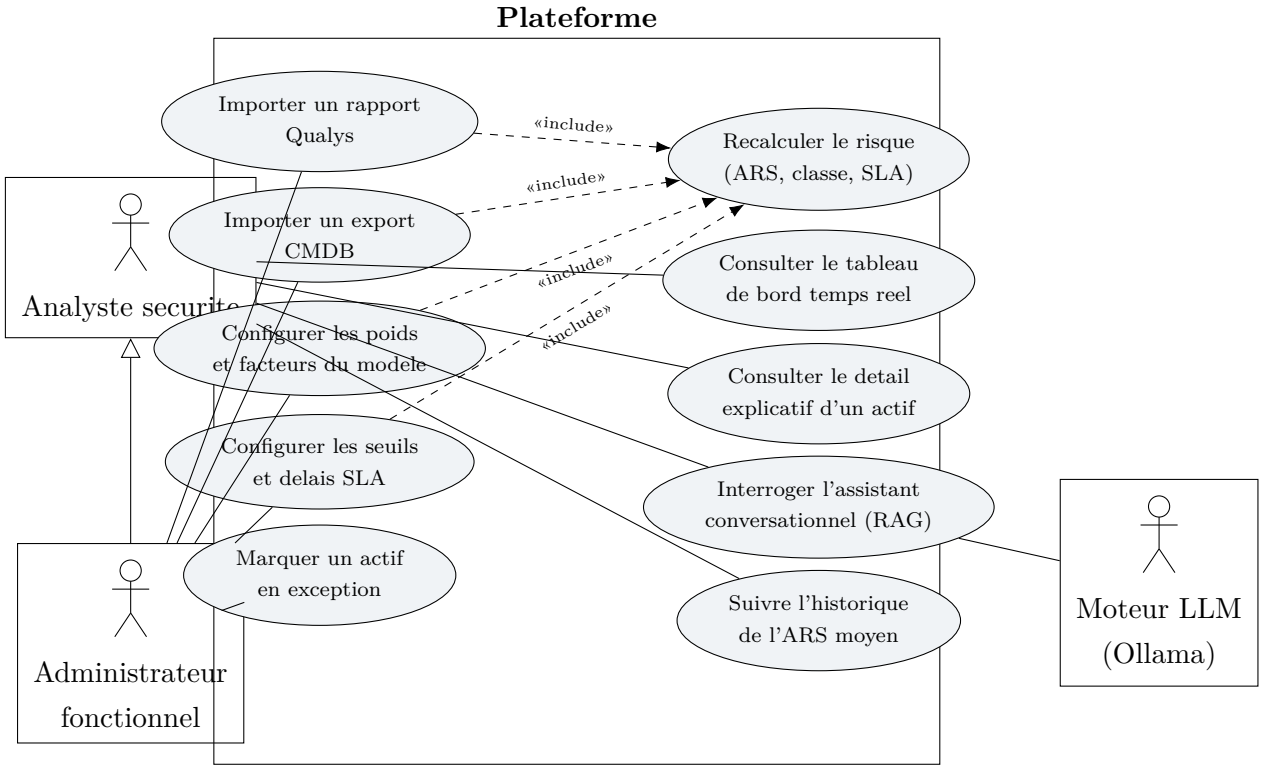


FIGURE 2.1 – Diagramme de cas d'utilisation de la plateforme

2.5 Description textuelle des cas d'utilisation principaux

2.5.1 UC – Importer un rapport Qualys / un export CMDB

Acteur principal	Administrateur fonctionnel
Pre-condition	Le fichier est un document XML dont le contenu correspond au format attendu (rapport Qualys ou export CMDB).
Scenario nominal	1) L'administrateur televerse le fichier via l'interface d'upload. 2) Le systeme detecte le type MIME reel du fichier et calcule son empreinte SHA. 3) Si l'empreinte correspond a un fichier deja traite, l'import est rejete. 4) Sinon, le fichier est stocke sur disque et marque comme en attente de traitement.

	5) L'administrateur declenche le lancement du traitement par lots (*batch*) correspondant au type de fichier. 6) Le systeme vide les donnees precedentes de ce type de source, puis lit, transforme et persiste les nouvelles donnees par lots. 7) A l'issue du traitement, un evenement d'ingestion terminee est publie, declenchant le recalcul du risque (cas d'utilisation inclus).
Post-condition	Les donnees de la source concernee refletent integralement le nouvel export ; le risque de tous les actifs impactes est recalcule.
Exceptions	Fichier deja traite (empreinte identique) ; type MIME incoherent avec le type declare ; traitement deja en cours (verrou applicatif).

2.5.2 UC – Consulter le tableau de bord temps reel

Acteur principal	Analyste securite
Scenario nominal	1) L'analyste ouvre la page du tableau de bord. 2) Le systeme calcule et renvoie l'ensemble des agregats (repartition par classe, par equipe, flux Sankey, historique, points SLA). 3) Le navigateur ouvre une connexion evenementielle (SSE) vers le serveur. 4) A chaque recalcul de risque abouti, le serveur pousse les nouveaux agregats a tous les clients connectes, qui redessinent les graphiques sans rechargement de page.
Post-condition	L'analyste dispose d'une vue a jour de la posture de risque du parc.

2.5.3 UC – Interroger l'assistant conversationnel (RAG)

Acteur principal	Analyste securite

Acteur secondaire	Moteur LLM local (Ollama)
Scenario nominal	1) L'analyste pose une question en langage naturel (ex. : « quels sont les actifs en classe Black non couverts par le SOC ? »). 2) L'agent conversationnel determine si la question necessite d'interroger la base de donnees via un outil expose (*tool calling*). 3) Si oui, l'outil est invoque, son resultat est injecte dans le contexte du modele. 4) Le modele local genere une reponse en flux (*streaming*) restituee progressivement a l'utilisateur.
Post-condition	L'analyste obtient une reponse fondee sur les donnees reelles du systeme plutot que sur la seule connaissance generale du modele.

3

Conception et architecture du systeme

3.1 Architecture globale

La plateforme est une application web monolithique modulaire, organisee en couches, et deployee sous forme de conteneurs Docker orchestres par Docker Compose. Le figure 3.1 presente l'architecture de deploiement retenue : un conteneur applicatif Spring Boot, une base de donnees relationnelle PostgreSQL, et un moteur d'inference local (Ollama) charge du modele de langage utilise par l'assistant RAG. Un conteneur ephemere (*ollama-init*) telecharge automatiquement le modele au premier demarrage, garantissant un environnement reproductible sans dependance a un service LLM externe payant.

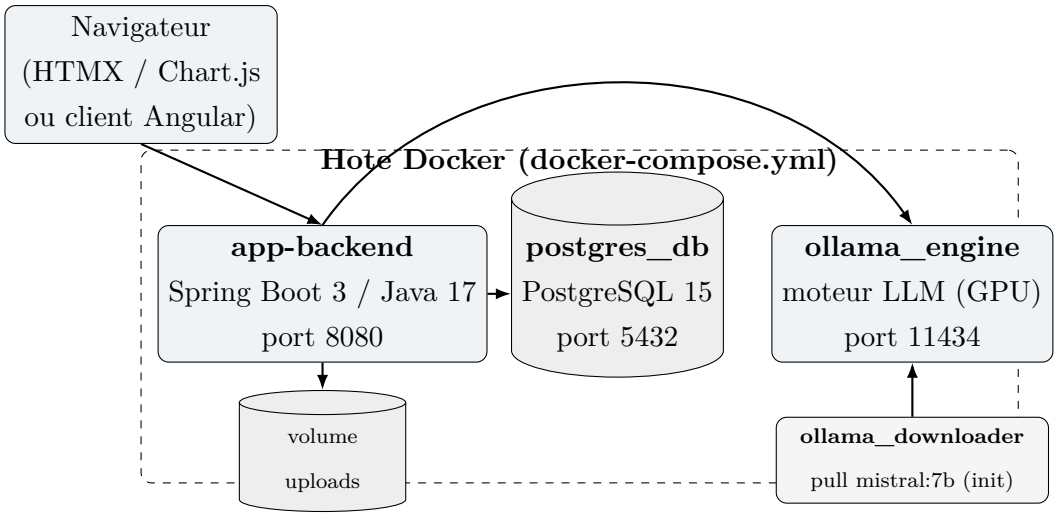


FIGURE 3.1 – Diagramme de deploiement (conteneurs Docker Compose)

Au sein du conteneur applicatif, le code est organise selon une architecture en couches classique (figure 3.2) : les *controllers* exposent les points d'entree HTTP (vues Thymeleaf/HTMX pour l'interface, endpoints REST pour le flux SSE et l'API), les *services* portent la logique metier (calcul du risque, agregation des metriques, orches-

tration des jobs Spring Batch), les *repositories* (Spring Data JPA) assurent la persistance, et les *entities* modelisent le domaine. Des *DTO* et des *mappers* (MapStruct) isolent le modele de domaine des formats d'entree/sortie (XML, JSON, formulaires HTML). La communication entre composants faiblement couples (fin d'ingestion, soumission d'une configuration, fin de recalcul) transite par un bus d'evenements applicatif (`ApplicationEventPublisher`), ce qui decouple le declencheur d'un recalcul de son execution effective.

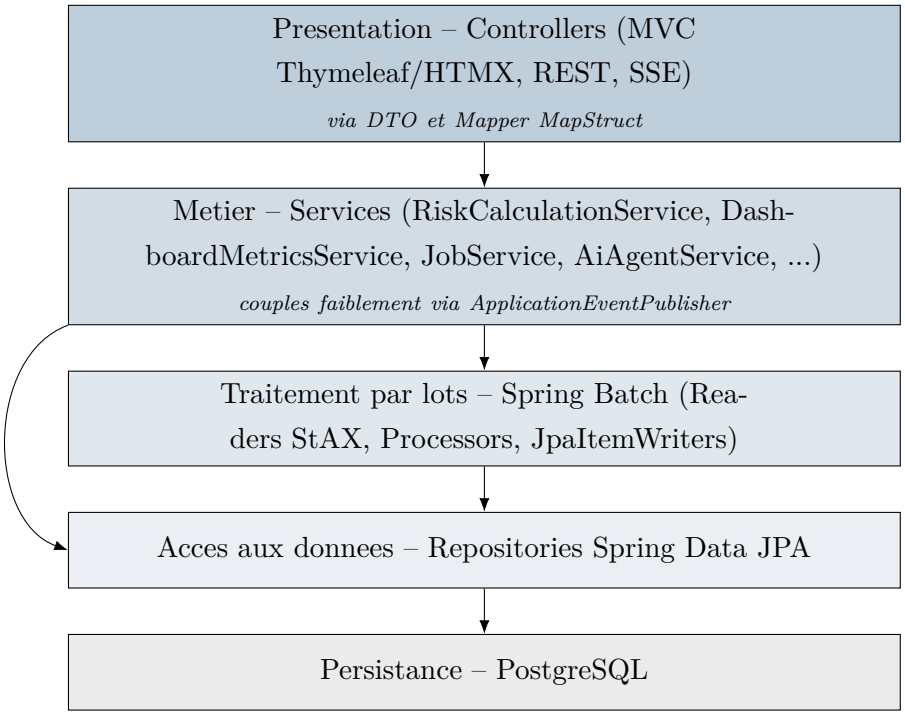


FIGURE 3.2 – Architecture logicielle en couches

3.2 Modele de donnees – diagramme de classes

Le figure 3.3 presente le coeur du modele de domaine. Un `Host` (hote scanne par Qualys) porte une collection de `VulnInfo` (relation de composition : la suppression d'un hote entraine celle de ses vulnerabilites, `CascadeType.ALL`). Un `CmdbCi` (element de configuration CMDB) n'est pas relie a `Host` par une cle etrangere, mais par une *reconciliation applicative* sur l'adresse IP, effectuee au moment du calcul de risque – ce choix, discute en section 3.2.1, permet d'ingerer independamment les deux sources sans contrainte d'integrite referentielle rigide entre deux systemes externes de rythmes de mise a jour differents. `RiskConfig` est une configuration singleton portant les parametres fixes de la formule (poids de severite, facteurs de modulation, seuils SLA) ; `RiskWeightConfig` est une table generique `(categorie, cle) -> poids` qui porte les poids dont l'ensemble des cles est variable (roles techniques, environnements, classes de risque pour le MTTR). `RiskSnapshot` historise, a chaque recalcul, la repartition

du parc par classe et l'ARS moyen, alimentant le graphique d'historique du tableau de bord.

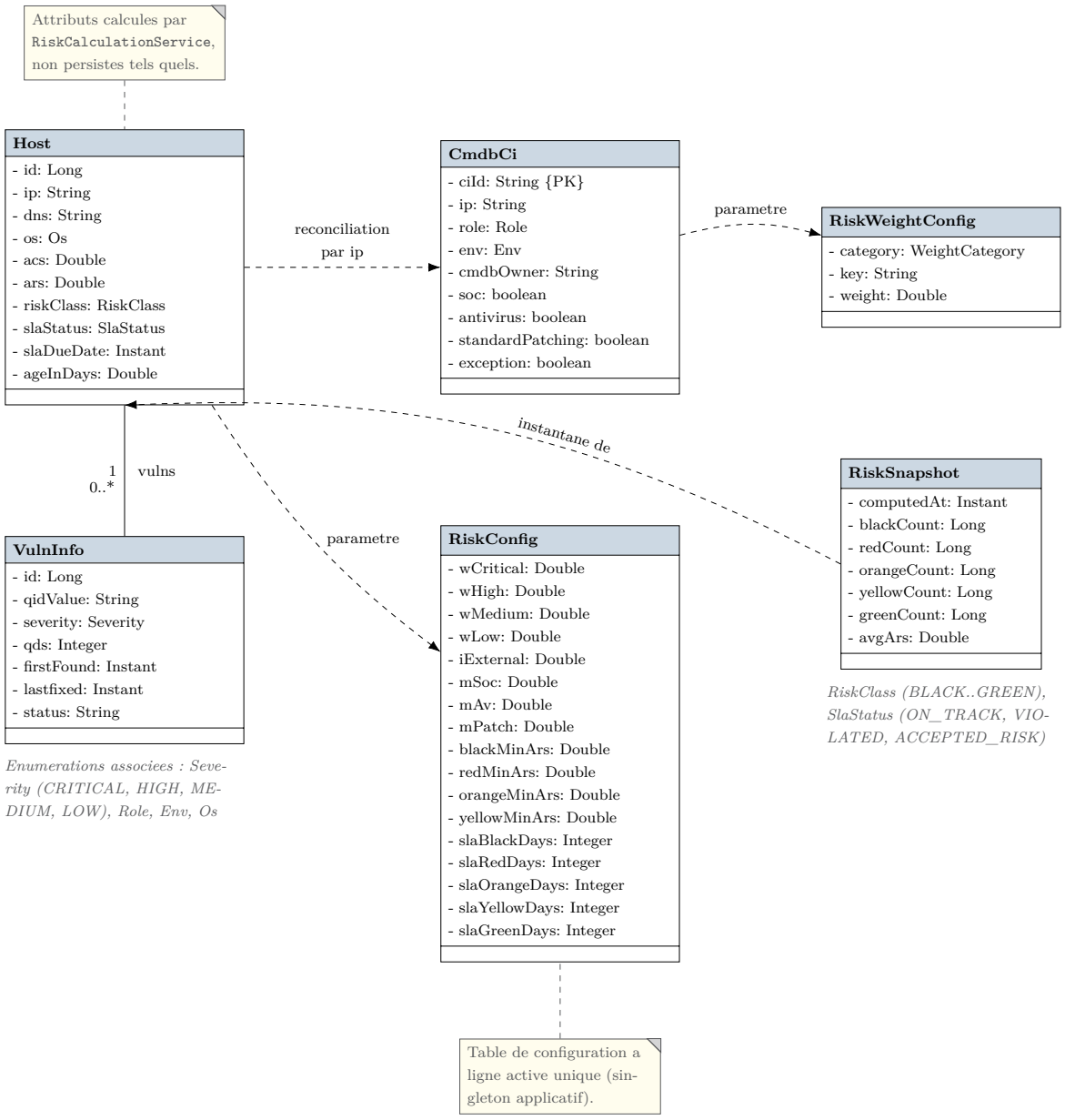


FIGURE 3.3 – Diagramme de classes simplifie du domaine

3.2.1 Le choix de la reconciliation par cle applicative

Qualys et la CMDB sont deux sources gerees par des equipes et des rythmes distincts : un hote peut avoir ete scanne sans encore figurer dans la CMDB (ou inversement). Modeliser une cle etrangere stricte entre `Host` et `CmdbCi` aurait oblige a rejeter ou differer l'ingestion de l'une des deux sources en cas d'incoherence temporaire. Le choix retenu – reconcilier les deux entites par adresse IP au moment du calcul (`Map<String, CmdbCi>` indexee par IP dans `RiskCalculationService`) – decouple totalement les deux pipelines d'ingestion et degrade proprement : un hote sans CI correspondant

n'est simplement pas score (il est compte comme *Shadow IT*, cf. chapitre 6).

3.3 Diagrammes de sequence

3.3.1 Ingestion d'un export et recalcul du risque

Le figure 3.4 decrit le scenario le plus important du systeme : de l'upload d'un fichier a la mise a jour du tableau de bord chez tous les clients connectes. Deux caracteristiques architecturales sont visibles sur ce diagramme :

- **L'asynchronisme a deux niveaux** : le stockage du fichier (`@Async`) et le lancement du job (`@Async`) rendent la main immediatement a l'appelant HTTP, evitant de bloquer un thread de requete pendant un traitement potentiellement long.
- **Le decouplage par evenements apres commit** : le recalcul du risque et la diffusion SSE ne sont pas appeles directement – ils reagissent a des evenements (`IngestionCompletedEvent`, `RiskRecalculationCompletedEvent`) publies et consommes en phase `AFTER_COMMIT`, garantissant que le listener ne lit jamais un etat de base de donnees non encore valide par la transaction qui l'a produit.

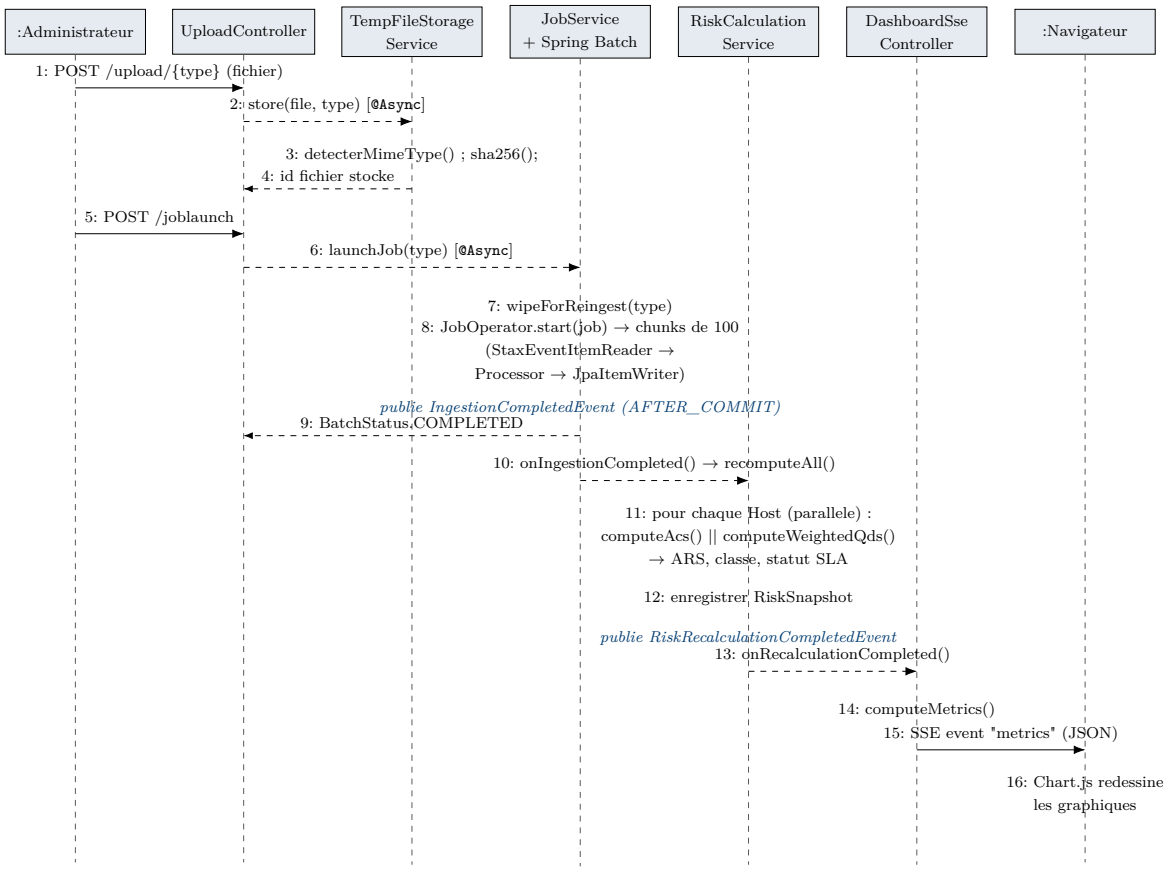


FIGURE 3.4 – Diagramme de sequence – ingestion d'un export et recalcul du risque

3.3.2 Consultation de l'assistant conversationnel

Le second flux notable est celui de l'assistant RAG. Contrairement a une architecture RAG classique qui recupere des passages de texte par similarite vectorielle avant generation, l'agent implemente ici s'appuie sur du *tool calling* : le modele decide lui-meme, en fonction de la question, quel outil interroger (par exemple « lister les actifs en classe Black »), l'outil execute une requete reelle sur la base de donnees, et son resultat est reinjecte dans le contexte avant generation de la reponse finale, diffusee en flux (`Flux<String>`) au fur et a mesure de sa production par le modele local.

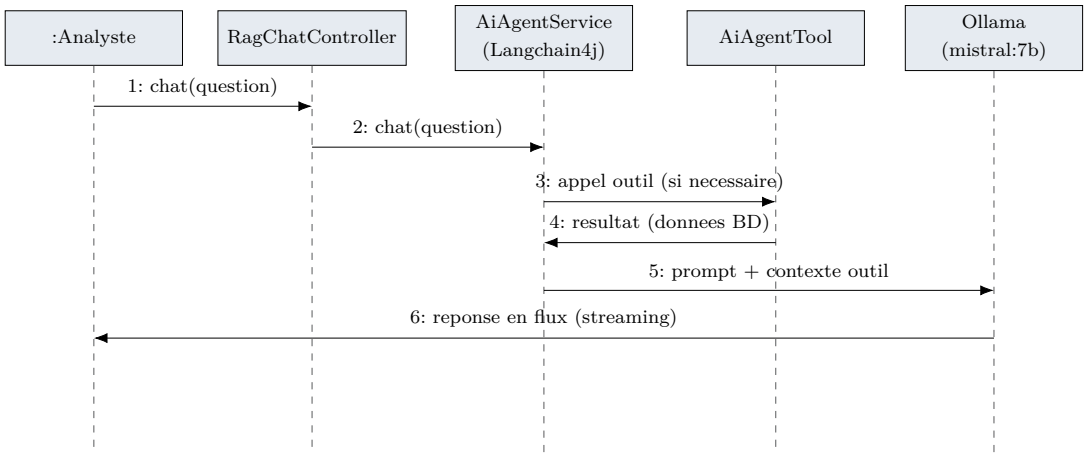


FIGURE 3.5 – Diagramme de sequence – interrogation de l'assistant RAG

3.4 Choix technologiques

Composant	**Technologie retenue**
Langage / plate-forme	Java 17, Spring Boot
Persistance	Spring Data JPA (Hibernate), PostgreSQL 15
Ingestion des fichiers	Spring Batch (traitement par blocs de 100 elements), lecture XML par flux via StAX (`StaxEventItemReader`) et marshalling JAXB (`Jaxb2Marshaller`, Spring OXM)
Detection de type de fichier	Apache Tika (verification du type MIME reel, independante de l'extension declaree)
Mapping objet	MapStruct (DTO $\leftrightarrow$ entites)
Asynchronisme	`@Async` sur des `Executor` dedies (E/S vs. calcul), `CompletableFuture`, bus d'evenements Spring (`ApplicationEventPublisher`, `@TransactionalEventListener`)
Interface utilisateur	Rendu cote serveur Thymeleaf, interactions dynamiques HTMX (en-tete `HX-Request` pour ne renvoyer que le fragment de page pertinent)
Tableau de bord	Server-Sent Events (`SseEmitter`) cote serveur, Chart.js cote client (avec le plugin de diagramme de Sankey)
Assistant conversationnel	Langchain4j (`@AiService`, tool calling), Ollama comme moteur d'inference local (modele `mistral:7b`)

Composant	**Technologie retenue**
Conteneurisation	Docker, Docker Compose (backend, base de donnees, moteur LLM)

4 Modelisation mathematique du risque

Ce chapitre formalise le modele de risque implemente par `RiskCalculationService` et `DashboardMetricsService`. Chaque formule est d'abord posee, puis accompagnee de proprietes demontrees qui justifient les choix de conception (bornage, monotonie, coherence de la classification) et qui mettent en evidence, lorsque c'est le cas, les ecarts assumes entre la specification initiale (*pfa_formule.pdf*) et son implementation.

4.1 Notations

Symbole	Signification
h	un actif (hote) reconcilie entre Qualys et la CMDB
$ACS(h)$	Asset Criticality Score de h, $ACS(h) \in [1, 5]$
$i \in \{c, h, m, l\}$	indice de severite : Critique, Elevee (*High*), Moyenne, Faible
w_i	poids configure de la severite i
QDS	Qualys Detection Score d'une vulnerabilite, $QDS \in [0, 100]$
$\text{Avg}(QDS_i)$	moyenne des QDS des vulnerabilites ouvertes de severite i sur h
n_i	nombre de vulnerabilites de severite i sur h
$I_{External}$	facteur d'exposition reseau (Internet)
M_{SOC}, M_{AV}, M_{Patch}	facteurs de modulation lies aux controles compensatoires
$ARS(h)$	Asset Risk Score de h
N	nombre total d'actifs actifs et scannes
TTR	temps de resolution (*Time To Remediate*) d'une vulnerabilite corrigee, en jours

Symbole	Signification
MTTR_w	temps moyen de remediation pondere par classe de risque
ERS	Enterprise Risk Score

4.2 Le score de criticite d'actif (ACS)

4.2.1 Definition

L'ACS traduit l'impact metier d'un actif independamment de ses vulnerabilites : un routeur de coeur de reseau en production n'a pas la meme importance qu'un serveur de test. Il est deduit de deux poids configures par l'administrateur – un poids par role technique $\rho : \mathrm{Role} \to [0,1]$ et un poids par environnement $\varepsilon : \mathrm{Env} \to [0,1]$ – moyennes puis recalees sur l'intervalle $[1,5]$:

$$ACS(h) = 1 + 4 \cdot \underbrace{\frac{\rho(\mathrm{role}(h)) + \varepsilon(\mathrm{env}(h))}{2}}_{\bar{x}(h)\ \in\ [0,1]} \tag{4.1}$$

Propriete 4.1 (Unicite du recalage affine)**.** *La fonction $f(x) = 1 + 4x$ est l'unique fonction affine $f(x) = ax+b$ verifiant $f(0) = 1$ et $f(1) = 5$, et elle envoie bijectivement $[0,1]$ sur $[1,5]$ en preservant l'ordre.*

Démonstration. Une fonction affine est entierement determinee par ses valeurs en deux points distincts. Les conditions aux bornes imposent $f(0) = b = 1$, puis $f(1) = a\cdot 1 + b = 5 \Rightarrow a = 4$: le couple $(a,b) = (4,1)$ est donc l'unique solution, d'ou $f(x) = 1 + 4x$. Comme $a = 4 > 0$, f est strictement croissante, donc injective sur $[0,1]$; combinee a $f(0) = 1$ et $f(1) = 5$, elle realise une bijection croissante de $[0,1]$ sur $[1,5]$. En particulier, pour tous $x, x' \in [0,1]$, $x \leq x' \iff f(x) \leq f(x')$: l'ordre des criticites relatives entre actifs est exactement preserve par le recalage. □

Cette propriete justifie le choix d'implementation : comparer deux ACS revient exactement a comparer les moyennes $\bar{x}$ sous-jacentes, aucune information d'ordre n'est perdue ni deformee par le recalage.

4.3 Le score de vulnerabilite pondere par severite

4.3.1 Formule de reference

La specification (eq. 1, terme interieur) pondere la moyenne des QDS de chaque classe de severite par un poids fixe :

$$\text{Score}_{\text{ref}}(h) = \sum_{i \in \{c,h,m,l\}} w_i \cdot \text{Avg}(QDS_i) \tag{4.2}$$

avec, valeurs de reference, $w_c = 1.0$, $w_h = 0.2$, $w_m = 0.05$, $w_l = 0.01$.

4.3.2 Raffinement implemente

L'implementation (`RiskCalculationService.computeWeightedQds`) introduit un terme correctif lie au *nombre* de vulnerabilites de chaque severite, afin qu'une classe comportant davantage de failles ouvertes pese un peu plus lourd, sans pour autant laisser une severite anecdotique (une seule faille isolee) ou au contraire un tres grand nombre de failles de meme severite faire exploser le score de maniere disproportionnee :

$$\text{WeightedQDS}(h) = \sum_{i \in \{c,h,m,l\}} w_i \cdot \text{Avg}(QDS_i) \cdot n_i^{1/100} \tag{4.3}$$

(convention : le terme vaut 0 si $n_i = 0$.)

Propriete 4.2 (Le facteur de comptage est croissant mais quasi borne en pratique)**.** *Pour $n \geq 1$, la fonction $\varphi(n) = n^{1/100}$ est strictement croissante et verifie $\varphi(1) = 1$. De plus, pour tout $n \leq 10^4$ (soit dix mille vulnerabilites d'une meme severite sur un meme hote, un majorant tres large en pratique), $\varphi(n) \leq 1{,}0965$.*

Démonstration. φ est derivable sur $(0, \infty)$ avec $\varphi'(n) = \frac{1}{100} n^{-99/100} > 0$, donc φ est strictement croissante ; $\varphi(1) = 1^{1/100} = 1$. Etant croissante, $\varphi(n) \leq \varphi(10^4)$ pour tout $n \leq 10^4$. Or

$$\varphi(10^4) = (10^4)^{1/100} = 10^{4/100} = 10^{0{,}04} = e^{0{,}04 \ln 10} = e^{0{,}09210\ldots} \approx 1{,}0965.$$

□

Corollaire 4.3 (Le raffinement domine strictement un multiplicateur brut)**.** *Remplacer $n_i^{1/100}$ par un multiplicateur brut n_i (comme le suggererait une ponderation « poids × moyenne × nombre ») fait croitre le terme lineairement et sans borne avec n_i, alors que propriete 4.2 montre que $n_i^{1/100}$ reste inferieur a 1,1 meme pour n_i de l'ordre du millier. Par exemple, pour $n_i = 1000$: le multiplicateur brut vaut 1000 (le terme est multiplie par mille) contre $1000^{1/100} = 10^{0{,}03} \approx 1{,}0715$ (augmentation de 7% seulement) avec le raffinement implemente.*

Ce corollaire justifie formellement le commentaire de conception du code source (*« gives classes with more open vulnerabilities a slowly growing extra push, without letting a single stray finding swing the term »*) : le nombre d'occurrences influence le

score de maniere monotone (propriete 4.2), donc coherente avec l'intuition « plus de failles ouvertes = plus de risque », mais son effet est strictement borne en pratique (propriete 4.3), de sorte que ni une faille isolee ni une accumulation massive de failles de faible impact ne peuvent, a elles seules, piloter le score.

4.4 L'Asset Risk Score (ARS)

4.4.1 Equation fondamentale

$$ARS(h) = \min\Big(ACS(h) \times \text{WeightedQDS}(h) \times I_{External}(h) \times M_{Controls}(h) \ , \ 1000 \Big) \tag{4.4}$$

avec le facteur de controles compensatoires :

$$M_{Controls}(h) = M_{SOC}(h) \times M_{AV}(h) \times M_{Patch}(h) \tag{4.5}$$

Chaque facteur de modulation est applique de maniere *conditionnelle*, avec 1 (element neutre de la multiplication) comme valeur par defaut lorsque sa condition d'activation n'est pas remplie :

$$I_{External}(h) = \begin{cases} 1{,}2 & \text{si } h \text{ expose sur Internet (QID 45004)} \\ 1{,}0 & \text{sinon} \end{cases} \tag{4.6}$$

$$M_{SOC}(h) = \begin{cases} 0{,}8 & \text{si supervise SOC} \\ 1{,}0 & \text{sinon} \end{cases} \tag{4.7}$$

$$M_{AV}(h) = \begin{cases} 0{,}9 & \text{si EDR/antivirus present} \\ 1{,}0 & \text{sinon} \end{cases} \tag{4.8}$$

$$M_{Patch}(h) = \begin{cases} 1{,}3 & \text{si exclu du cycle de patch standard} \\ 1{,}0 & \text{sinon} \end{cases} \tag{4.9}$$

Le tableau 4.2 recapitule ces valeurs de reference (table 1 de la specification).

TABLE 4.2 – Matrice des multiplicateurs de risque residuel

Facteur	Source	Condition	Valeur
$I_{External}$	Qualys (QID 45004)	si expose sur Internet	1,2
M_{SOC}	CMDB	si actif monitore (SIEM)	0,8
M_{AV}	CMDB	si presence d'un EDR/antivirus	0,9
M_{Patch}	CMDB	si exclusion du cycle de patch	1,3

4.4.2 Positivite et monotonie

Propriete 4.4 (Positivite)**.** *Pour tout h, $ARS(h) \in [0, 1000]$.*

Démonstration. $ACS(h) \geq 1 > 0$ par propriete 4.1. WeightedQDS(h) est une somme de termes $w_i \cdot \text{Avg}(QDS_i) \cdot n_i^{1/100}$ tous positifs ou nuls (poids $w_i \geq 0$, $QDS \geq 0$, et $n_i^{1/100} \geq 0$ ou terme conventionnellement nul si $n_i = 0$), donc WeightedQDS(h) ≥ 0. $I_{External}(h)$ et $M_{Controls}(h)$ sont des produits de constantes strictement positives, donc strictement positifs. Le produit $ACS(h) \times \text{WeightedQDS}(h) \times I_{External}(h) \times M_{Controls}(h)$ est donc positif ou nul, et $\min(\cdot, 1000)$ le majore par 1000. □

Propriete 4.5 (Monotonie)**.** *$ARS(h)$ est une fonction (faiblement) croissante de chacun de ses quatre facteurs ACS,* WeightedQDS*, $I_{External}$, $M_{Controls}$, les trois autres etant fixes.*

Démonstration. Notons $P = ACS \times \text{WeightedQDS} \times I_{External} \times M_{Controls}$ le produit avant saturation, et $g(x) = \min(x, 1000)$. Soient $a \leq a'$ deux valeurs possibles pour l'un des quatre facteurs (les trois autres, notes $k \geq 0$, etant fixes) : alors $P(a) = k \cdot a \leq k \cdot a' = P(a')$ car $k \geq 0$. La fonction g est elle-meme croissante : pour $x \leq x'$, $\min(x, 1000) \leq \min(x', 1000)$ (cas immediat par disjonction $x' \leq 1000$ ou $x' > 1000$). La composee d'une fonction croissante d'une variable ($P(\cdot)$ dans la direction consideree) avec une fonction croissante (g) est croissante, donc $ARS = g \circ P$ est croissante en chacun de ses facteurs. □

Cette propriete garantit que le modele ne peut jamais produire de contre-intuition grave : ajouter une mesure de protection (facteur < 1), reduire l'exposition ($I_{External} = 1$ au lieu de 1,2), ou reduire la gravite des vulnerabilites ne peut jamais *augmenter* l'ARS affiche.

4.4.3 Condition de saturation

Le plafond a 1000 evite qu'un score sans borne ne perde toute signification au-dela du pire cas possible. On peut caracteriser precisement quand il s'active.

Propriete 4.6 (Seuil de saturation)**.** *Avec les valeurs de reference du tableau 4.2 et* $ACS(h) \leq 5$*, on a* $ACS(h) \times I_{External}(h) \times M_{Controls}(h) \leq 5 \times 1{,}2 \times 1{,}3 = 7{,}8$*. Par consequent, si* $\text{WeightedQDS}(h) < \frac{1000}{7{,}8} \approx 128{,}2$*, alors* $ARS(h)$ *n'est* ***pas*** *sature, et* $ARS(h)$ *est exactement egal au produit équation (4.4) (le minimum n'intervient pas).*

Démonstration. $M_{Controls}(h) = M_{SOC}(h)M_{AV}(h)M_{Patch}(h)$ est maximise lorsque le terme protecteur M_{SOC} et M_{AV} valent chacun 1 (absence de protection, valeur neutre, superieure a 0,8 et 0,9) et que le terme aggravant M_{Patch} vaut 1,3 (exclusion du cycle de patch) : $M_{Controls}(h) \leq 1 \times 1 \times 1{,}3 = 1{,}3$. Avec $ACS(h) \leq 5$ et $I_{External}(h) \leq 1{,}2$, le produit des trois facteurs est majore par $5 \times 1{,}2 \times 1{,}3 = 7{,}8$. Si $\text{WeightedQDS}(h)$ est tel que $7{,}8 \times \text{WeightedQDS}(h) < 1000$, alors, a fortiori, le produit complet équation (4.4) (majore par $7{,}8 \times \text{WeightedQDS}(h)$) est strictement inferieur a 1000, donc $\min(\cdot, 1000)$ vaut l'argument lui-meme. □

Une application numerique de cette propriete est fournie au chapitre 7 (section 7.2).

4.5 Analyse de sensibilite : la hierarchie des severites

Propriete 4.7 (Ratio de contribution entre severites)**.** *A moyenne de QDS et nombre de vulnerabilites egaux entre deux severites i et j (*$\text{Avg}(QDS_i) = \text{Avg}(QDS_j)$ *et* $n_i = n_j$*), le rapport des termes correspondants dans équation (4.3) est exactement egal au rapport des poids de severite :*

$$\frac{terme_i}{terme_j} = \frac{w_i}{w_j}.$$

Avec les valeurs de reference, une vulnerabilite Critique pese donc ***5 fois*** *plus qu'une vulnerabilite Elevee (*$w_c/w_h = 1{,}0/0{,}2 = 5$*),* ***20 fois*** *plus qu'une vulnerabilite Moyenne (*$w_c/w_m = 1{,}0/0{,}05 = 20$*), et* ***100 fois*** *plus qu'une vulnerabilite Faible (*$w_c/w_l = 1{,}0/0{,}01 = 100$*).*

Démonstration. Par definition, $\text{terme}_i = w_i \cdot \text{Avg}(QDS_i) \cdot n_i^{1/100}$ et $\text{terme}_j = w_j \cdot \text{Avg}(QDS_j) \cdot n_j^{1/100}$. Sous l'hypothese $\text{Avg}(QDS_i) = \text{Avg}(QDS_j)$ et $n_i = n_j$, ces deux facteurs communs se simplifient dans le quotient, laissant $\text{terme}_i/\text{terme}_j = w_i/w_j$. Les valeurs numeriques s'obtiennent en substituant $w_c = 1{,}0$, $w_h = 0{,}2$, $w_m = 0{,}05$, $w_l = 0{,}01$. □

Cette propriete est la traduction mathematique exacte de l'intention metier du modele : concentrer l'attention operationnelle sur les vulnerabilites critiques, independamment de leur nombre relatif, puisque le rapport de poids est invariant et ne depend jamais des donnees observees.

4.6 Classification en classes de risque et gouvernance SLA

4.6.1 Classification

L'ARS, une fois calcule, est projete sur cinq classes ordonnees par des seuils configures $\tau_B > \tau_R > \tau_O > \tau_Y \geq 0$ (valeurs de reference dans le tableau 4.3) :

$$\text{Classe}(ARS) = \begin{cases} \text{BLACK} & \text{si } ARS \geq \tau_B \\ \text{RED} & \text{si } \tau_R \leq ARS < \tau_B \\ \text{ORANGE} & \text{si } \tau_O \leq ARS < \tau_R \\ \text{YELLOW} & \text{si } \tau_Y \leq ARS < \tau_O \\ \text{GREEN} & \text{si } ARS < \tau_Y \end{cases} \tag{4.10}$$

TABLE 4.3 – Matrice dynamique des SLA de remediation (valeurs de reference)

Classe	**Seuil ARS**	**SLA exige**	**Couleur**
Black	$ARS \geq 700$	2 jours	Noir
Red	$600 \leq ARS < 700$	7 jours	Rouge
Orange	$400 \leq ARS < 600$	15 jours	Orange
Yellow	$200 \leq ARS < 400$	30 jours	Jaune
Green	$ARS < 200$	90 jours	Vert

Propriete 4.8 (Bonne definition de la classification)**.** *Si $\tau_B > \tau_R > \tau_O > \tau_Y \geq 0$, alors équation (4.10) definit une fonction totale sur $[0, 1000]$: tout $ARS \in [0, 1000]$ appartient a exactement une classe.*

Démonstration. Exhaustivite. Soit $ARS \in [0, 1000]$. Si $ARS < \tau_Y$, il est classe GREEN. Sinon $ARS \geq \tau_Y$; s'il est de plus $< \tau_O$, il est classe YELLOW. En poursuivant ainsi, tout $ARS \geq \tau_Y$ non classe YELLOW/ORANGE/RED tombe necessairement dans le dernier cas $ARS \geq \tau_B$ (BLACK), car les quatre cas couvrent respectivement $(-\infty, \tau_Y)$, $[\tau_Y, \tau_O)$, $[\tau_O, \tau_R)$, $[\tau_R, \tau_B)$ et $[\tau_B, +\infty)$, dont la reunion est $\mathbb{R}$ tout entier des lors que $\tau_Y \leq \tau_O \leq \tau_R \leq \tau_B$. *Exclusion mutuelle.* Ces cinq intervalles sont deux a deux disjoints car chacun est de la forme $[\tau_k, \tau_{k+1})$ avec les τ_k strictement ordonnes (ou une demi-droite pour les cas extremes), donc un reel ne peut appartenir qu'a un seul d'entre eux. Une valeur ARS ne peut donc satisfaire qu'une seule des cinq conditions de équation (4.10), ce qui prouve que la classification est une fonction bien definie (et non une relation multivaluee). □

Corollaire 4.9 (Coherence ordinale)**.** *Si $ARS_1 \leq ARS_2$, la classe de ARS_1 n'est jamais strictement plus severe que celle de ARS_2, pour l'ordre naturel $GREEN < YELLOW < ORANGE < RED < BLACK$.*

Démonstration. Consequence immediate de équation (4.10) : chaque classe correspond a un intervalle de ARS, et ces intervalles sont ranges dans le meme ordre que la severite des classes ($GREEN$ pour les plus petites valeurs, $BLACK$ pour les plus grandes). La fonction Classe est donc une fonction en escalier croissante de ARS vers l'echelle ordinale des classes. □

C'est cette coherence qui garantit qu'augmenter le risque technique d'un actif ne peut jamais faire baisser sa classe de gouvernance – une propriete indispensable pour la confiance des equipes dans le systeme.

4.6.2 Gestion des exceptions (Risk Acceptance)

Certains actifs ne peuvent etre patches dans les delais pour des raisons de disponibilite metier ou d'incompatibilite logicielle. La CMDB porte alors un attribut booleen `Exception`. Son activation **ne modifie pas** l'ARS (le risque technique reste reel) mais suspend le decompte SLA standard :

$$\text{StatutSLA}(h) = \begin{cases} \text{ACCEPTED_RISK} & \text{si Exception}(h) = \text{oui} \\ \text{VIOLATED} & \text{si non exceptee et delai depasse} \\ \text{ON_TRACK} & \text{sinon} \end{cases} \tag{4.11}$$

ou $\text{date_echeance}(h) = \text{premiere_detection_ouverte}(h) + \text{SLA}_{\text{jours}}(\text{Classe}(ARS(h)))$, le nombre de jours de SLA etant lu dans le tableau 4.3 selon la classe de l'actif.

4.7 Mesure de la performance : le MTTR pondere

4.7.1 Definition

Pour un ensemble de vulnerabilites corrigees, regroupees par classe de risque k de l'actif auquel elles appartenaient, avec des poids de performance w_k (configurables, valeurs de reference $w_{black} = 10$, $w_{red} = 5$, ...) :

$$\text{MTTR}_w = \frac{\sum_k w_k \sum_{v \in \text{corrigees}_k} TTR(v)}{\sum_k w_k \cdot C_k} \tag{4.12}$$

ou C_k est le nombre de vulnerabilites corrigees de la classe k, et la somme ne porte que sur les actifs **sans exception active** (équation (4.11)), conformement au principe de ne pas penaliser les equipes operationnelles pour des blocages decides au niveau metier.

Propriete 4.10 (Le MTTR pondere est une moyenne ponderee des MTTR par classe)**.** *En notant* $\mathrm{MTTR}_k = \frac{1}{C_k}\sum_{v\in corrigees_k} TTR(v)$ *la moyenne simple des temps de resolution de la classe* k *(pour* $C_k > 0$*), on a*

$$\mathrm{MTTR}_w = \sum_k \lambda_k \,\mathrm{MTTR}_k, \qquad \lambda_k = \frac{w_k C_k}{\sum_j w_j C_j} \geq 0, \qquad \sum_k \lambda_k = 1,$$

et par consequent

$$\min_k \mathrm{MTTR}_k \;\leq\; \mathrm{MTTR}_w \;\leq\; \max_k \mathrm{MTTR}_k.$$

Démonstration. Par definition de MTTR_k, $\sum_{v\in\mathrm{corrigees}_k} TTR(v) = C_k \cdot \mathrm{MTTR}_k$. En substituant dans le numerateur de équation (4.12) :

$$\mathrm{MTTR}_w = \frac{\sum_k w_k\, C_k\, \mathrm{MTTR}_k}{\sum_k w_k C_k} = \sum_k \underbrace{\frac{w_k C_k}{\sum_j w_j C_j}}_{\lambda_k} \mathrm{MTTR}_k.$$

Chaque $\lambda_k \geq 0$ car $w_k \geq 0$ et $C_k \geq 0$, et $\sum_k \lambda_k = \frac{\sum_k w_k C_k}{\sum_j w_j C_j} = 1$: $(\lambda_k)_k$ est donc une distribution de poids convexes, et MTTR_w est une combinaison convexe des MTTR_k. Toute combinaison convexe de reels est comprise entre leur minimum et leur maximum : en notant $m = \min_k \mathrm{MTTR}_k$ et $M = \max_k \mathrm{MTTR}_k$, on a $m \leq \mathrm{MTTR}_k \leq M$ pour tout k, donc $m = \sum_k \lambda_k\, m \leq \sum_k \lambda_k \,\mathrm{MTTR}_k \leq \sum_k \lambda_k\, M = M$. □

Cette propriete est essentielle pour l'interpretation de l'indicateur : le MTTR_w affiche au comite de securite ne peut jamais etre une valeur aberrante hors de la plage des performances reellement observees par classe ; il en est une moyenne recentree sur les classes les plus critiques (celles qui recoivent le plus grand poids w_k).

4.8 Agregation globale : l'Enterprise Risk Score (ERS)

4.8.1 Definition

Pour le pilotage strategique, les *ARS* individuels sont agreges en un seul indicateur, module par un facteur de performance operationnelle globale :

$$ERS = \left(\frac{1}{N}\sum_{i=1}^{N} ARS_i\right) \times \mathrm{Global_I_{Ops}} \tag{4.13}$$

$$\text{Global_I}_{\text{Ops}} = \begin{cases} 1{,}00 & \text{si } \text{MTTR}_{global} \leq \text{SLA}_{cible} \\ 1{,}15 & \text{si } \text{SLA}_{cible} < \text{MTTR}_{global} \leq 1{,}5 \cdot \text{SLA}_{cible} \\ 1{,}30 & \text{si } \text{MTTR}_{global} > 1{,}5 \cdot \text{SLA}_{cible} \end{cases} \tag{4.14}$$

Propriete 4.11 (Monotonie du facteur de penalite)**.** *Pour* $\text{SLA}_{cible} > 0$ *fixe,* $\text{Global_I}_{\text{Ops}}$ *est une fonction (faiblement) croissante de* MTTR_{global}*.*

Démonstration. Les trois cas de équation (4.14) partitionnent $[0, +\infty)$ en trois intervalles $\left(0, \text{SLA}_{cible}\right]$, $\left(\text{SLA}_{cible}, 1{,}5\,\text{SLA}_{cible}\right]$ et $\left(1{,}5\,\text{SLA}_{cible}, +\infty\right)$, disjoints et ordonnes de gauche a droite. Sur chaque intervalle, la fonction est constante, donc trivialement croissante. Aux deux points de raccord, la valeur saute de 1,00 a 1,15 puis de 1,15 a 1,30 : la fonction ne decroit jamais lorsqu'on passe d'un intervalle au suivant. Une fonction en escalier dont les paliers sont croissants de gauche a droite est globalement croissante au sens large sur son domaine. □

Corollaire 4.12 (Monotonie de l'ERS en performance operationnelle)**.** *Pour une moyenne d'ARS fixee, l'ERS est une fonction croissante de* MTTR_{global} *: plus les equipes de remediation prennent du retard par rapport a la cible SLA, plus l'indicateur global remonte a la direction est penalise, independamment de toute amelioration ou degradation du risque technique brut.*

Démonstration. ERS est le produit d'une constante positive ($\frac{1}{N}\sum ARS_i$, fixee par hypothese) par $\text{Global_I}_{\text{Ops}}(\text{MTTR}_{global})$, qui est croissante par propriete 4.11. Le produit d'une constante strictement positive par une fonction croissante est croissant. □

Exemple 4.13. Si la moyenne technique du parc est $\overline{ARS} = 400$ (classe Orange) mais que le MTTR_{global} mesure depasse 1,5 fois la cible SLA (l'IT met systematiquement deux fois plus de temps que prevu a corriger), alors $\text{Global_I}_{\text{Ops}} = 1{,}30$ et

$$ERS = 400 \times 1{,}30 = 520,$$

ce qui fait franchir a l'indicateur affiche a la direction la frontiere de la classe Orange vers la classe Red (tableau 4.3), reveletant une paralysie operationnelle que la seule moyenne technique aurait masquee.

4.8.2 Ecart entre specification et implementation actuelle

A la date de redaction de ce rapport, le service `DashboardMetricsService` calcule l'agregat expose au tableau de bord comme une simple moyenne arithmetique des ARS (équation (4.13) sans le facteur $\text{Global_I}_{\text{Ops}}$) : aucune cible SLA globale n'est encore configuree ni comparee au MTTR_w mesure pour declencher la penalite équation (4.14).

Le modele mathematique complet est donc entierement specifie et demontre ci-dessus, mais son cablage dans le service de calcul reste, a ce stade, une extension a realiser – elle est reprise comme perspective d'evolution prioritaire (section 7.4).

4.9 Synthese

TABLE 4.4 – Recapitulatif des formules du modele de risque

Grandeur	**Formule**
$ACS(h)$	$1 + 4 \cdot \frac{\rho(\text{role}) + \varepsilon(\text{env})}{2}$
WeightedQDS(h)	$\sum_i w_i \cdot \text{Avg}(QDS_i) \cdot n_i^{1/100}$
$ARS(h)$	$\min\left(ACS \times \text{WeightedQDS} \times I_{External} \times M_{Controls}\,,\, 1000\right)$
Classe(ARS)	seuils $\tau_B > \tau_R > \tau_O > \tau_Y$ (équation (4.10))
MTTR_w	$\frac{\sum_k w_k \sum TTR_k}{\sum_k w_k C_k}$ (hors actifs en exception)
ERS	$\left(\frac{1}{N}\sum ARS_i\right) \times \text{Global_I}_{\text{Ops}}(\text{MTTR}_{global})$

5 Realisation et implementation

5.1 Environnement technique

La plateforme est developpee en Java 17 sur Spring Boot, avec PostgreSQL comme systeme de persistance et Ollama comme moteur d'inference local pour l'assistant conversationnel. L'ensemble est conteneurise avec Docker et orchestre par Docker Compose (figure 3.1), ce qui permet de reproduire l'environnement complet – base de donnees, backend, moteur LLM et telechargement automatique du modele – sur n'importe quelle machine disposant de Docker.

5.2 Pipeline d'ingestion des donnees

5.2.1 Reception et securisation du fichier

Un fichier depose par l'utilisateur passe par trois controles avant d'etre accepte (liste 5.1) : detection du type MIME reel par Apache Tika (independamment de l'extension declaree, pour eviter qu'un fichier malveillant deguise en `.xml` ne soit traite), calcul d'une empreinte cryptographique, et rejet si cette empreinte correspond a un fichier deja traite. Ce dernier point repond directement au probleme identifie en phase de cadrage (*app_notes_dxc.txt*, section 4) : sans deduplication, un utilisateur qui reuploade par erreur le meme export declencherait un nouveau traitement par lots inutile.

```
1 String digest = computeHash(file);
2 if (fileRepository.existsByHash(digest)) {
3     throw new FileProcessedException();
4 }
5 fileRepository.deleteByName(type.name().toLowerCase());
6 fileRepository.save(new File(UUID.randomUUID(), null, file.getSize(),
7         detectMimeType(file), Instant.now(), type.name().toLowerCase()
              ,
8         digest, false));
```

Listing 5.1 – Deduplication par empreinte avant stockage (TempFileStorageServiceimpl)

5.2.2 Traitement par lots (Spring Batch)

Le traitement effectif est confie a Spring Batch. Chaque type de source (Qualys, CMDB) possede son propre *job*, compose d'une seule etape orientee bloc (*chunk*) de 100 elements : un lecteur `StaxEventItemReader` parcourt le XML en flux (sans le charger integralement en memoire) en isolant les fragments correspondant a l'element racine attendu (`HOST` pour Qualys, `row` pour la CMDB), un *processor* transforme chaque DTO en entite via un *mapper* MapStruct, et un `JpaItemWriter` persiste le lot.

```
@Bean
public Step cmdbStep(JobRepository jobRepository,
        MultiResourceItemReader<CmdbDto> reader,
        PlatformTransactionManager tx,
        JpaItemWriter<CmdbCi> writer,
        CmdbProcessor processor) {
    return new StepBuilder(jobRepository)
            .<CmdbDto, CmdbCi>chunk(100)
            .transactionManager(tx)
            .reader(reader)
            .processor(processor)
            .writer(writer)
            .build();
}
```

Listing 5.2 – Etape Spring Batch d'ingestion CMDB (BatchConfig)

Chaque nouvel import est traite comme la verite complete et actuelle de sa source : avant de lancer le job, `JobService.wipeForReingest` supprime l'integralite des lignes precedentes du type concerne (`Host` pour Qualys, en cascade sur ses `VulnInfo` ; `CmdbCi` pour la CMDB). Ce choix (documente en section 1.4) evite qu'un actif retire du parc ne reste indefiniment visible dans le tableau de bord, au prix de l'absence d'historique detaille des CI entre deux imports – une limite assumee et reprise en section 7.4.

Le lancement d'un job est protege par un verrou applicatif (`ReentrantLock`) : les *readers* StAX sont resolus tardivement via les parametres du job (liaison par contexte de thread), et deux jobs executes en parallele pourraient faire lire a l'un d'eux le motif de fichiers de l'autre. Le verrou garantit qu'un seul job d'ingestion s'execute a la fois.

5.2.3 Recalcul du risque : parallelisme et coherence transactionnelle

Une fois l'ingestion terminee, le recalcul de risque (`RiskCalculationService.recomputeAll`) traite chaque hote reconcilie de maniere independante et parallele : pour chaque actif, le calcul de l'ACS et celui du score QDS pondere sont soumis a un pool de threads dedie au calcul (`CALC_EXECUTOR`), puis combines des que les deux resultats sont disponibles.

```
CompletableFuture<Double> acsFuture =
    CompletableFuture.supplyAsync(() -> computeAcs(cmdbCi, roleWeights
        , envWeights), calcExecutor);
CompletableFuture<Double> weightedQdsFuture =
    CompletableFuture.supplyAsync(() -> computeWeightedQds(host,
        config), calcExecutor);

return acsFuture.thenCombine(weightedQdsFuture,
    (acs, weightedQds) -> applyRiskScore(host, cmdbCi, config, acs,
        weightedQds));
```

Listing 5.3 – Parallelisation du calcul par actif (RiskCalculationService)

Le declenchement du recalcul n'est jamais appele directement par le controleur d'upload : il reagit a un evenement applicatif (`IngestionCompletedEvent`), consomme par un *listener* transactionnel configure en phase `AFTER_COMMIT`. Ce choix, deja illustre au figure 3.4, evite une classe de bug subtile mais reelle : sans cette precaution, le listener pourrait demarrer sa propre transaction de lecture avant que la transaction d'ecriture des nouvelles donnees ne soit effectivement validee, et recalculer le risque sur un etat de base partiellement ecrit.

5.3 Tableau de bord temps reel

Le tableau de bord combine deux mecanismes : un point d'entree REST classique (`GET /api/v1/dashboard/metrics`) qui calcule et renvoie l'ensemble des agregats a l'ouverture de la page, et un flux Server-Sent Events (`GET /api/v1/dashboard/stream`) auquel le navigateur reste connecte. A chaque fin de recalcul de risque, le meme calcul d'agregats est effectue une seule fois cote serveur et pousse a tous les clients connectes, qui redessinent leurs graphiques Chart.js sans recharger la page (figure 3.4, etapes 13 a 16). Ce choix evite le double ecueil du *polling* (requetes repetees inutiles) et du rechargement manuel.

5.4 Assistant conversationnel (RAG)

Ce module reste, au moment de la redaction de ce rapport, au stade du developpement et de la conception : il s'agit d'un prototype fonctionnel demontrable, qui n'a pas encore fait l'objet d'un deploiement en environnement de recette (*staging*) ni d'une campagne de validation comparable a celle menee sur le moteur de risque (chapitre 7).

L'assistant s'appuie sur Langchain4j (`@AiService`) et sur un modele executes localement via Ollama (`mistral:7b`), plutot que sur une API LLM externe payante – un choix coherent avec la sensibilite des donnees manipulees (inventaire de vulnerabilites et de criticites metier). L'architecture retenue privilegie le *tool calling* : plutot que d'indexer les donnees dans une base vectorielle et de faire de la recherche par similarite, l'agent dispose d'outils qui executent des requetes reelles sur la base au moment ou la question l'exige, garantissant que les reponses portent sur l'etat courant des donnees et non sur un instantane potentiellement perime.

```
@AiService
public interface AiAgentService {
    @SystemMessage("""
            You are an assistant with access to tools that query a
               live database.
            When a question can be answered by a tool, you MUST invoke
                the tool.
            Never describe, explain or write out the call - actually
               perform it.
            Never output code.""")
    Flux<String> chat(String question);
}
```

Listing 5.4 – Definition de l'agent conversationnel (AiAgentService)

5.5 Conteneurisation et deploiement

Le fichier `docker-compose.yml` definit quatre services : la base PostgreSQL (avec *healthcheck* pour garantir que le backend ne demarre qu'une fois la base disponible), le backend Spring Boot, le moteur Ollama (avec reservation GPU pour l'inference), et un conteneur ephemere `ollama-init` charge de telecharger automatiquement le modele au premier demarrage. Un volume nomme persiste les fichiers uploades (`/app/uploads`) independamment du cycle de vie du conteneur applicatif.

6 Visualisation des donnees : lecture et interpretation des graphiques

Le tableau de bord de la plateforme (page `/dashboard`, alimentee par `DashboardMetricsService` et rafraichie par le flux SSE decrit au chapitre 5) restitue la posture de risque du parc a travers cinq graphiques complementaires, chacun repondant a une question de pilotage differente. Ce chapitre reproduit chacun d'eux avec des **donnees illustratives** (le tableau de bord reel affiche les donnees issues des imports Qualys/CMDB) et explique comment le lire.

6.1 Repartition par classe de risque

Question a laquelle il repond : *quelle proportion du parc se trouve dans chaque classe de risque ?*

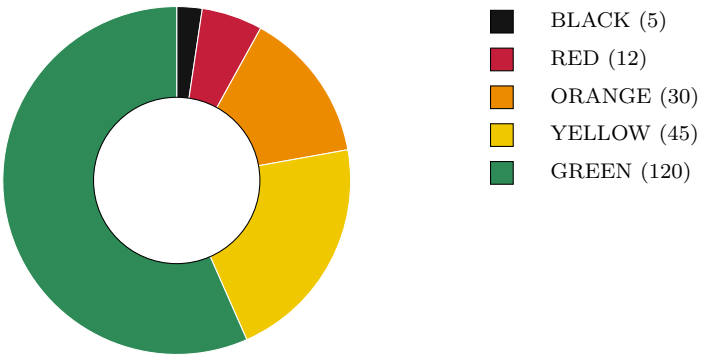


FIGURE 6.1 – Repartition des actifs par classe de risque (diagramme en anneau, donnees illustratives)

Comment le lire. Chaque secteur represente une classe de risque (tableau 4.3) ; sa taille angulaire est proportionnelle au nombre d'actifs qui y appartiennent, et sa couleur suit le code couleur officiel de la matrice SLA (noir, rouge, orange, jaune, vert). Un coup d'oeil suffit pour evaluer la sante globale du parc : un anneau majoritairement vert

avec des secteurs noir/rouge fins indique un parc globalement maitrise avec un nombre restreint d'urgences a traiter – c'est la lecture attendue sur l'exemple ci-dessus, ou les classes BLACK et RED ne representent ensemble que $17/212 \approx 8\%$ du parc.

6.2 Repartition par equipe proprietaire

Question a laquelle il repond : *quelle equipe porte le plus de risque critique a traiter ?*

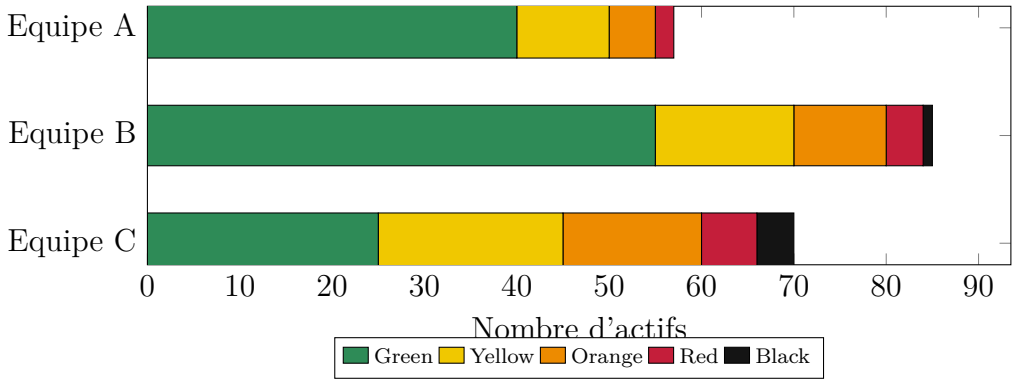


FIGURE 6.2 – Repartition des actifs par equipe proprietaire (barres horizontales empilees, donnees illustratives)

Comment le lire. Chaque ligne horizontale correspond au proprietaire CMDB (`cmdbOwner`) d'un ensemble d'actifs ; sa longueur totale est le nombre d'actifs qu'elle possede, et chaque segment colore est le nombre d'actifs de cette equipe dans une classe de risque donnee. Contrairement au graphique en anneau (vision globale), ce graphique permet de **cibler l'action** : sur l'exemple, l'Equipe C possede moins d'actifs au total que l'Equipe B, mais une proportion bien plus grande en classes Orange/Red/Black – c'est elle qui devrait etre prioritairement sollicitee par le comite de securite.

6.3 Flux Environnement → Role → Classe de risque

Question a laquelle il repond : *d'ou vient le risque, structurellement – quels environnements et quels roles techniques concentrent les actifs les plus critiques ?*

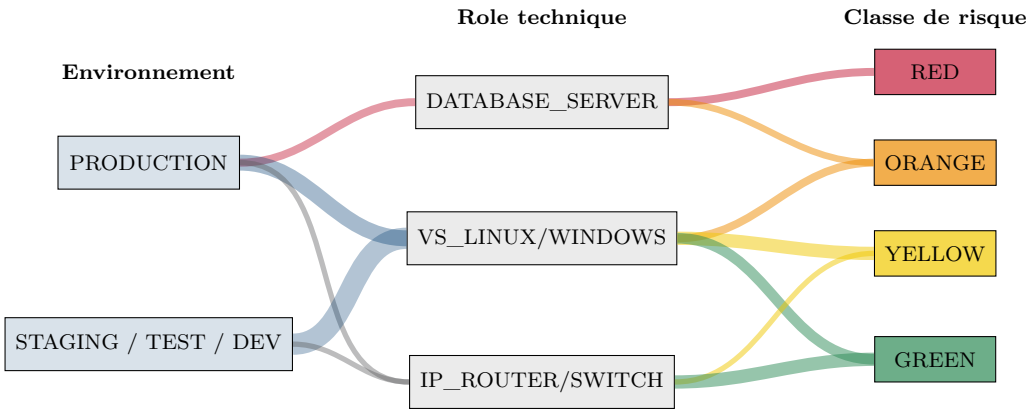


FIGURE 6.3 – Diagramme de flux (Sankey) Environnement → Role → Classe de risque (donnees illustratives)

Comment le lire. Chaque bande horizontale relie deux categories ; sa **largeur (l'epaisseur du trait) est proportionnelle au nombre d'actifs** empruntant ce chemin, et sa couleur reprend soit l'environnement d'origine, soit – pour le second niveau – la classe de risque de destination (c'est exactement la convention utilisee par `DashboardMetricsService`, qui construit ce graphique en deux temps : `Env→Role` d'abord, `Role→RiskClass` ensuite). On lit ce graphique de gauche a droite en suivant l'epaisseur des flux : sur l'exemple, les bases de donnees de production concentrent un flux epais vers RED, signalant qu'il s'agit du couple (role, environnement) le plus critique du parc – une information que ni le graphique en anneau ni le graphique par equipe ne permettent de voir directement, puisqu'aucun des deux ne croise simultanement l'environnement et le role technique.

6.4 Historique de la repartition du parc

Question a laquelle il repond : *la posture de risque du parc s'ameliore-t-elle ou se degrade-t-elle au fil des campagnes de scan successives ?*

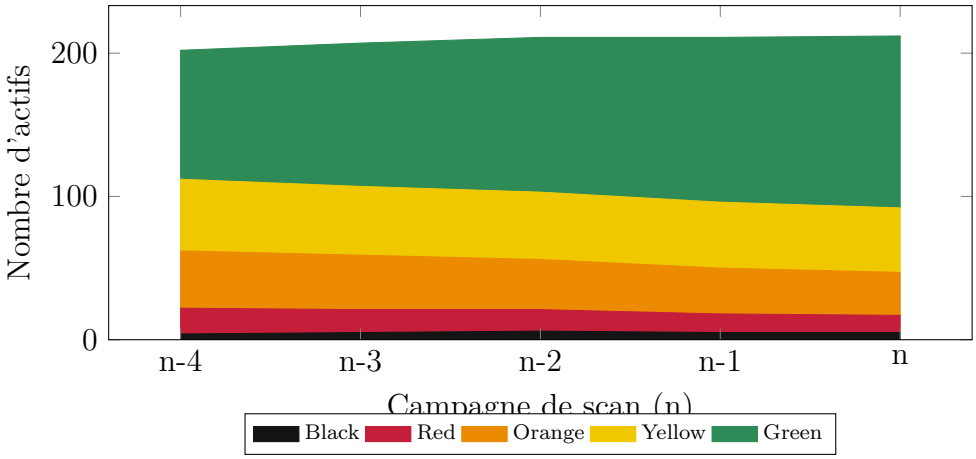


FIGURE 6.4 – Historique empile de la repartition par classe (donnees illustratives, une serie par instantane `RiskSnapshot`)

Comment le lire. Chaque bande coloree empilee represente l'effectif d'une classe de risque a un instant donne (un `RiskSnapshot` enregistre a chaque recalcul, chapitre 3) ; la hauteur totale de la pile est la taille du parc scanne a ce moment-la. Ce n'est pas la hauteur d'une bande prise isolement qui importe, mais son **evolution relative dans le temps** : sur l'exemple, la bande verte (GREEN) s'epaissit progressivement tandis que les bandes orange et rouge s'amincissent – signe d'une tendance favorable, meme si le nombre d'actifs en classe Black reste globalement stable, ce qui justifierait une attention particuliere portee a ce sous-ensemble specifique plutot qu'a la tendance globale.

6.5 Conformite SLA (age vs. score de risque)

Question a laquelle il repond : *quels actifs sont en violation de leur delai de remediation, et de combien ?*

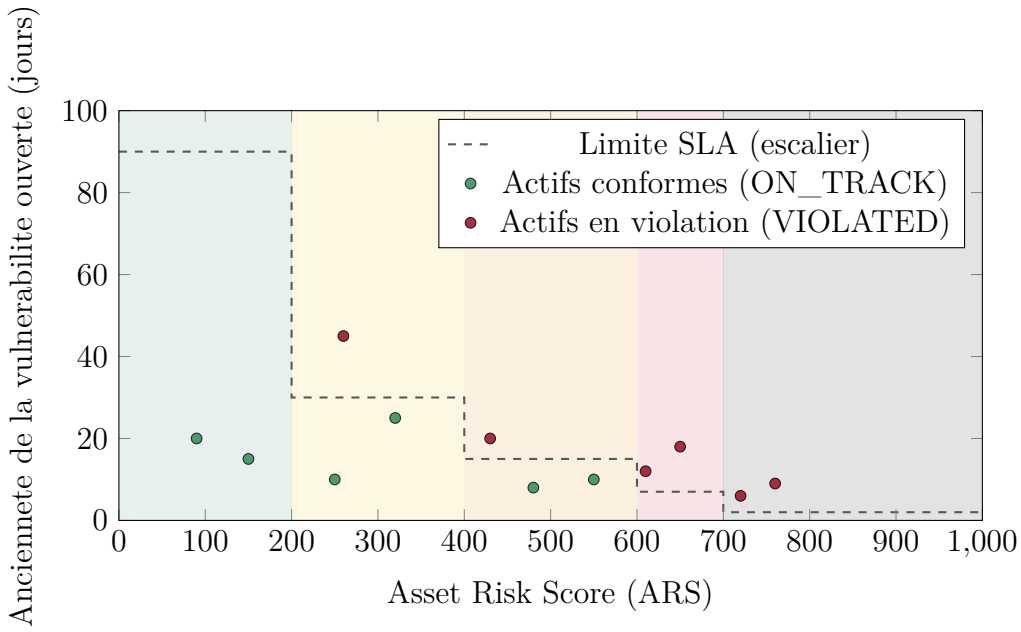


FIGURE 6.5 – Conformite SLA : anciennete vs. ARS, avec zones de classe en fond (donnees illustratives)

Comment le lire. Chaque point est un actif ; sa position horizontale est son ARS (équation (4.4)) et sa position verticale, l'age en jours de sa vulnerabilite ouverte la plus ancienne. Les bandes de fond delimitent les cinq classes de risque (tableau 4.3) et la ligne en escalier grise trace le delai SLA maximal autorise pour la classe correspondant a chaque plage d'ARS. **La regle de lecture est geometrique** : un point situe *au-dessus* de la ligne en escalier, dans sa propre bande de couleur, est en violation de son SLA – c'est exactement le critere programme dans `applyRiskScore` (équation (4.11)), rendu visuellement immediat. Sur l'exemple, le point a ($ARS = 260$, age $= 45$j) tombe dans la zone Yellow (SLA de 30 jours) alors qu'il affiche 45 jours d'anciennete : il est donc bien au-dessus de la marche de l'escalier a cet endroit, coherent avec sa couleur rouge (violation). A l'inverse, un point situe sous la ligne, dans sa bande, est encore *dans les temps*. Ce graphique est celui qui traduit le plus directement la gouvernance SLA du chapitre 4 en decision operationnelle : il permet d'identifier en un coup d'oeil les actifs qui necessitent une escalade immediate.

7 Tests, validation et resultats

7.1 Strategie de test

Le projet integre les quatre dependances de test standard de l'ecosysteme Spring Boot couvrant respectivement la persistance, le web MVC, la securite et le traitement par lots (`data-jpa`, `webmvc`, `security`, `batch`), completees par AssertJ et Hamcrest pour l'expressivite des assertions. La strategie de test suit la structure en couches du figure 3.2 :

- **Tests unitaires** sur les services de calcul (`RiskCalculationService`, `Dashboard-MetricsService`) : verification des formules du chapitre 4 sur des jeux de donnees synthetiques controles, en particulier des cas limites (hote sans vulnerabilite, actif sans correspondance CMDB, ARS proche du seuil de saturation).
- **Tests d'integration Spring Batch**, permis par la dependance `batch-test`, pour valider que le pipeline d'ingestion (lecture StAX → mapping → ecriture JPA) produit bien les entites attendues a partir d'un extrait XML de reference.
- **Tests de mapping** (MapStruct) verifiant que chaque champ du DTO issu du parsing XML est correctement reporte sur l'entite JPA correspondante.

7.2 Validation numerique du modele ARS

Pour illustrer et verifier concretement les proprietes demontrees au chapitre 4, cette section deroule le calcul complet sur un actif fictif mais realiste.

Exemple 7.1 (Calcul complet de l'ARS pour un serveur de base de donnees de production)**.** Soit un hote h correspondant, dans la CMDB, a un CI de role `DATABASE_SERVER` en environnement `PRODUCTION`, supervise par le SOC, sans EDR installe, et exclu du cycle de patch standard (maintenance fenetre critique). Qualys a detecte sur cet hote

2 vulnerabilites Critiques (QDS moyen 92), 3 vulnerabilites Elevees (QDS moyen 78) et 5 vulnerabilites Moyennes (QDS moyen 55), et l'une des vulnerabilites correspond au QID 45004 (exposition Internet).

1. ACS. Avec des poids configures $\rho(\text{DATABASE_SERVER}) = 0{,}9$ et $\varepsilon(\text{PRODUCTION}) = 1{,}0$:

$$ACS(h) = 1 + 4 \cdot \frac{0{,}9 + 1{,}0}{2} = 1 + 4 \times 0{,}95 = 4{,}8.$$

2. WeightedQDS (équation (4.3), poids de reference) :

$$\begin{aligned}
\text{terme}_c &= 1{,}0 \times 92 \times 2^{1/100} = 92 \times 1{,}00696 \approx 92{,}64 \\
\text{terme}_h &= 0{,}2 \times 78 \times 3^{1/100} = 15{,}6 \times 1{,}01108 \approx 15{,}77 \\
\text{terme}_m &= 0{,}05 \times 55 \times 5^{1/100} = 2{,}75 \times 1{,}01630 \approx 2{,}79 \\
\text{terme}_l &= 0 \quad \text{(aucune vulnerabilite Faible)} \\
\text{WeightedQDS}(h) &\approx 92{,}64 + 15{,}77 + 2{,}79 = 111{,}20
\end{aligned}$$

Conformement a la propriete 4.7, on verifie que le terme Critique domine largement : il represente a lui seul 92,64/111,20 $\approx$ 83% du score, bien que ne comptant que 2 vulnerabilites sur 10.

3. Facteurs de modulation (tableau 4.2) : exposition Internet averee, $I_{External} = 1{,}2$; SOC present, $M_{SOC} = 0{,}8$; pas d'EDR, $M_{AV} = 1{,}0$; exclu du cycle de patch, $M_{Patch} = 1{,}3$:

$$M_{Controls}(h) = 0{,}8 \times 1{,}0 \times 1{,}3 = 1{,}04.$$

4. ARS (équation (4.4)) :

$$ARS(h) = \min\Big(4{,}8 \times 111{,}20 \times 1{,}2 \times 1{,}04\ ,\ 1000\Big) = \min(666{,}5\,;\,1000) = 666{,}5.$$

Comme prevu par la propriete 4.6 (seuil de saturation $\approx$ 128,2 pour WeightedQDS, tres inferieur au 111,20 obtenu ici mais du meme ordre de grandeur), l'actif s'approche de la saturation sans l'atteindre : le plafond de 1000 n'intervient pas, $ARS(h)$ est exactement le produit calcule.

5. Classification (équation (4.10), tableau 4.3) : $600 \leq 666{,}5 < 700$, donc h est classe **RED**, avec un SLA exige de **7 jours**.

6. Statut SLA. Si la vulnerabilite Critique la plus ancienne a ete detectee il y a 10 jours et que le CI n'est pas en exception, alors la date d'echeance ($10 - 7 = 3$ jours dans le passe par rapport a aujourd'hui, en supposant la detection il y a 10 jours et un SLA de 7 jours) est depassee : $\text{StatutSLA}(h) = \text{VIOLATED}$ (équation (4.11)). Sur le figure 6.5, cet actif se situerait au point (666,5 ; 10), au-dessus de la marche « 7 jours » de la zone RED : visuellement en violation, coherent avec le calcul.

7.3 Validation du MTTR pondere

Exemple 7.2. Sur une periode donnee, les vulnerabilites corrigees (hors actifs en exception) se repartissent ainsi : classe RED, $C_{red} = 4$ corrections, TTR moyen $\mathrm{MTTR}_{red} = 5$ jours ; classe ORANGE, $C_{orange} = 10$ corrections, $\mathrm{MTTR}_{orange} = 12$ jours. Avec les poids de performance de reference $w_{red} = 5$ et $w_{orange} = 3$:

$$\mathrm{MTTR}_w = \frac{w_{red}\, C_{red}\, \mathrm{MTTR}_{red} + w_{orange}\, C_{orange}\, \mathrm{MTTR}_{orange}}{w_{red}\, C_{red} + w_{orange}\, C_{orange}} = \frac{5 \times 4 \times 5 + 3 \times 10 \times 12}{5 \times 4 + 3 \times 10}$$
$$= \frac{100 + 360}{20 + 30} = \frac{460}{50} = 9{,}2 \text{ jours.}$$

Conformement a la propriete 4.10, $\mathrm{MTTR}_w = 9{,}2$ est bien compris entre $\min(5, 12) = 5$ et $\max(5, 12) = 12$; il est de plus plus proche de la valeur ORANGE que de la moyenne simple non ponderee $\frac{5+12}{2} = 8{,}5$ ne l'aurait laisse supposer, car le poids relatif effectif de la classe ORANGE ($w_{orange}C_{orange} = 30$) depasse celui de RED ($w_{red}C_{red} = 20$) malgre son poids de performance w individuel plus faible – consequence directe du volume corrige plus important sur cette classe.

7.4 Bilan des resultats obtenus

La plateforme realisee couvre de bout en bout le scenario cible : import d'un export Qualys et d'un export CMDB, calcul automatique et parallelise de l'ARS pour chaque actif reconcilie, classification et gouvernance SLA, historisation des instantanes de risque, restitution temps reel sous forme de cinq graphiques complementaires (chapitre 6), et un prototype d'interrogation en langage naturel via l'assistant RAG (chapitre 5), ce dernier restant au stade du developpement et de la conception et n'entrant pas dans le perimetre de la campagne de validation ci-dessus. Les proprietes mathematiques demontrees au chapitre 4 (bornage de l'ARS, monotonie, coherence de la classification, comportement en moyenne ponderee du MTTR) ont ete verifiees numeriquement sur les exemples ci-dessus, confirmant que l'implementation respecte fidelement le modele theorique specifie – a l'exception assumee et documentee du facteur $\mathrm{Global_I}_{\mathrm{Ops}}$ de l'ERS (section 4.8.2).

Conclusion generale et perspectives

Bilan

Ce projet de fin d'annee a permis de concevoir et de realiser une plateforme qui transforme deux sources de donnees fragmentees – exports Qualys et CMDB – en un score de risque unique, contextualise et gouverne par des delais de remediation explicites. Le travail a couvert l'ensemble de la chaine, de l'ingestion asynchrone par lots des fichiers XML jusqu'a la restitution temps reel sous forme de tableau de bord, en passant par la formalisation et la demonstration mathematique complete du modele de risque (chapitre 4) : bornage et monotonie de l'ARS, coherence de la classification en cinq classes, comportement du MTTR pondere comme moyenne convexe verifiable, et modelisation du facteur de penalite operationnelle Global_I_{Ops} de l'Enterprise Risk Score. Chacune de ces proprietes a ete verifiee numeriquement au chapitre 7 sur des exemples complets, confirmant la coherence entre la specification, sa preuve mathematique et son implementation.

Au-dela du moteur de calcul, le projet a egalement permis d'explorer des problematiques d'ingenierie logicielle a part entiere : traitement par lots robuste et idempotent avec Spring Batch, coherence transactionnelle d'une architecture evenementielle asynchrone, restitution temps reel via Server-Sent Events, et integration d'un assistant conversationnel fonde sur un modele de langage execute localement plutot que sur une dependance externe.

Limites du travail realise

Plusieurs limites, assumees dans le cadre du perimetre defini (section 1.4), meritent d'etre explicitement mentionnees :

- La chaine de securite HTTP autorise actuellement toute requete sans authenti-

fication (`permitAll`) : ce choix, adapte a un prototype fonctionnel demontrable, n'est pas compatible avec un usage en production sur des donnees aussi sensibles qu'un inventaire de vulnerabilites.

- La strategie d'ingestion *wipe-and-reingest* (section 5.2) simplifie considerablement le pipeline, mais empeche toute analyse d'evolution fine de l'inventaire CMDB lui-meme (ajout/retrait de CI) independamment du recalcul de risque.
- Le facteur Global_I_{Ops} de l'ERS, entierement specifie et demontre au chapitre 4, n'est pas encore cable dans `DashboardMetricsService` : l'indicateur affiche est aujourd'hui une simple moyenne des ARS (section 4.8.2).
- L'assistant conversationnel (RAG), decrit au chapitre 5, reste au stade du developpement et de la conception : c'est un prototype fonctionnel, non deploye en environnement de recette (*staging*) ni valide par une campagne de tests comparable a celle menee sur le moteur de risque (chapitre 7). Sa latence, identifiee des la phase de cadrage comme un point de vigilance, n'a en particulier pas fait l'objet d'un diagnostic instrumente permettant de determiner precisement la part imputable au modele, aux outils invoques ou aux acces base de donnees.

Perspectives

Plusieurs axes d'evolution decoulent directement du bilan precedent et du perimetre initialement ecarte :

1. **Completer l'agregation ERS** en cablant le facteur Global_I_{Ops} (équation (4.14)) : ceci suppose d'introduire une cible SLA globale configurable et de comparer le $MTTR_{global}$ mesure a cette cible a chaque recalcul, pour exposer aux comites de securite l'indicateur complet specifie.
2. **Introduire l'authentification et le controle d'acces par role (RBAC)**, distinguant a minima le role Administrateur (import, configuration) du role Analyste (consultation), en coherence avec la generalisation d'acteurs deja identifiee au chapitre 2.
3. **Historiser la CMDB** plutot que la remplacer integralement a chaque import, pour permettre une analyse de tendance sur l'inventaire des actifs eux-memes (croissance du parc, disparition d'actifs non decommissionnes proprement), en complement de l'historique d'ARS deja assure par `RiskSnapshot`.
4. **Renforcer la couverture de tests automatises** du moteur de risque, en particulier des cas limites identifies par les preuves du chapitre 4 (saturation de

l'ARS, absence de vulnerabilites, absence de correspondance CMDB, classes de risque aux bornes exactes des seuils).

5. **Optimiser la latence de l'assistant RAG**, notamment via un cache (Redis) des descriptions de CVE reutilisees d'une session a l'autre – piste deja identifiee lors du cadrage initial – et via une instrumentation permettant de distinguer le temps passe dans le modele, dans les outils et dans les acces base de donnees.

6. **Notifier proactivement** les equipes proprietaires (email, messagerie d'entreprise) des lors qu'un actif franchit le seuil de violation SLA (équation (4.11)), plutot que de s'appuyer uniquement sur la consultation active du tableau de bord.

7. **Elargir les sources ingerees** au-dela de Qualys et d'un format de CMDB unique, pour couvrir d'autres scanners de vulnerabilites et systemes de gestion de configuration frequemment rencontres chez les clients de DXC Technology.

8. **Produire des exports de restitution** (rapport PDF ou tableur) a destination des comites de securite periodiques, en particulier pour la revue des actifs places en exception (équation (4.11)), explicitement prevue par la gouvernance mais non encore outillee.

Ces perspectives ne remettent pas en cause le coeur du modele mathematique demontre dans ce rapport : elles en etendent la couverture operationnelle et consolident sa mise en production, dans la continuite directe du travail realise durant ce stage.

A

Extraits des formats de donnees en entree

A.1 Extrait d'un export CMDB (XML)

```xml
<row>
  <ciId>CI-00123</ciId>
  <dns>db-prod-01.exemple.local</dns>
  <ip>10.20.30.40</ip>
  <os>LINUX</os>
  <env>PRODUCTION</env>
  <role>DATABASE_SERVER</role>
  <cmdbOwner>Equipe DBA</cmdbOwner>
  <agentQualys>true</agentQualys>
  <soc>true</soc>
  <antivirus>false</antivirus>
  <standardPatching>false</standardPatching>
  <exception>false</exception>
</row>
```

Listing A.1 – Structure simplifiee d'une ligne d'export CMDB

A.2 Extrait d'un rapport Qualys (XML)

```xml
<HOST>
  <IP>10.20.30.40</IP>
  <DNS>db-prod-01.exemple.local</DNS>
  <VULN_INFO_LIST>
    <VULN_INFO>
      <QID>45004</QID>
      <SEVERITY>4</SEVERITY>
      <QDS>92</QDS>
      <FIRST_FOUND_DATETIME>2026-06-01T10:00:00Z</FIRST_FOUND_DATETIME
         >
      <LAST_FOUND_DATETIME>2026-08-20T10:00:00Z</LAST_FOUND_DATETIME>
```

```
11        <STATUS>Active</STATUS>
12      </VULN_INFO>
13    </VULN_INFO_LIST>
14  </HOST>
```

Listing A.2 – Structure simplifiee d'un hote dans un rapport Qualys

B

Parametres par defaut du modele de risque

Le tableau B.1 recapitule les valeurs par defaut portees par l'entite `RiskConfig`, utilisees tant qu'aucune configuration personnalisee n'a ete soumise par l'administrateur.

TABLE B.1 – Valeurs par defaut de la configuration du risque

Groupe	**Parametre**	**Valeur**
Poids de severite	$w_{critical}$	1,00
	w_{high}	0,20
	w_{medium}	0,05
	w_{low}	0,01
Facteurs de modulation	$I_{External}$	1,20
	M_{SOC}	0,80
	M_{AV}	0,90
	M_{Patch}	1,30
Seuils de classe (ARS min.)	Black	700
	Red	600
	Orange	400
	Yellow	200
Delais SLA (jours)	Black	2
	Red	7
	Orange	15
	Yellow	30
	Green	90

C

Glossaire

Terme	Definition
Actif (asset)	Element de configuration reconcilie entre un hote scanne par Qualys et une fiche CMDB, sur la base de son adresse IP.
Chunk (bloc)	Unite de traitement de Spring Batch : un lot d'elements lus, transformes puis ecrits atomiquement (ici, 100 elements).
Reconciliation	Rapprochement applicatif, effectue au moment du calcul de risque, entre un `Host` issu de Qualys et un `CmdbCi` issu de la CMDB partageant la meme adresse IP.
Shadow IT	Actif present dans une source (Qualys ou CMDB) sans correspondance dans l'autre, donc non reconcilie et non score.
Tool calling	Mecanisme par lequel un modele de langage declenche l'execution d'une fonction applicative (ici, une requete sur la base de donnees) plutot que de generer une reponse a partir de sa seule connaissance parametrique.

Bibliographie

[1] Qualys, Inc. *Qualys Detection Score (QDS) – Documentation technique*. https://www.qualys.com/.

[2] FIRST.org. *Common Vulnerability Scoring System (CVSS) v3.1 Specification Document*. https://www.first.org/cvss/.

[3] VMware / Spring Team. *Spring Batch Reference Documentation*. https://docs.spring.io/spring-batch/reference/.

[4] VMware / Spring Team. *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/.

[5] Langchain4j Project. *Langchain4j Documentation – AI Services and Tool Calling*. https://docs.langchain4j.dev/.

[6] Ollama. *Ollama Documentation – Run large language models locally*. https://ollama.com/.

[7] Chart.js Contributors. *Chart.js Documentation*. https://www.chartjs.org/docs/.

[8] NIST. *Special Publication 800-40 Rev. 4 – Guide to Enterprise Patch Management Planning*. National Institute of Standards and Technology, 2022.

[9] ISO/IEC. *ISO/IEC 27005:2022 – Information security, cybersecurity and privacy protection – Guidance on managing information security risks*.

[10] Documentation d'architecture interne. *Modelisation Mathematique du Risque et Gouvernance DevSecOps – Modele d'Evaluation TruRisk Etendu*. DXC Technology, 2026.